

Go Language Tutorial

1. 变量和常量

Go 语言变量名由字母、数字、下划线组成，其中首个字符不能为数字。Go 语言中关键字和保留字都不能用作变量名。

Go 语言中的变量需要声明后才能使用，同一作用域内不支持重复声明。并且 Go 语言的变量声明后必须使用。

1.1 变量

1.1.1 var 声明变量

```
1 | var identifier type
```

```
1 | var name string
2 | var age int
3 | var isOk bool
```

```
1 | package main
2 |
3 | import "fmt"
4 |
5 | func main() {
6 |     var username = "张三"
7 |     var age int = 20
8 |     fmt.Println(username, age)
9 | }
```

1.1.2 一次定义多个变量

```
1 | var identifier1, identifier2 type
```

```
1 | package main
2 |
3 | import "fmt"
4 |
5 | func main() {
6 |     var username, sex string
7 |     username = "张三"
8 |     sex = "男"
9 |     fmt.Println(username, sex)
10 | }
```

1.1.3 批量声明变量的时候指定类型

```
1  var (  
2      a string  
3      b int  
4      c bool  
5  )  
6  a = "张三"  
7  b = 10  
8  c = true  
9  fmt.Println(a,b,c)
```

1.1.4 变量的初始化

Go 语言在声明变量的时候，会自动对变量对应的内存区域进行初始化操作。每个变量会被初始化成其类型的默认值，例如：整型和浮点型变量的默认值为 `0`。字符串变量的默认值为空字符串。布尔型变量默认为 `false`。切片、函数、指针变量的默认为 `nil`。

当然我们也可在声明变量的时候为其指定初始值。 变量初始化的标准格式如下：

```
1  var name string = "zhangsan"  
2  var age int = 18
```

1.1.5 类型推导

有时候我们会将变量的类型省略，这个时候编译器会根据等号右边的值来推导变量的类型完成初始化。

```
1  var name = "zhangsan"  
2  var age = 18
```

1.1.6 短变量声明法

在函数内部，可以使用更简略的 `:=` 方式声明并初始化变量。

⚠ Caution

短变量只能用于声明局部变量， 不能用于全局变量的声明

```

1 package main
2
3 import "fmt"
4
5 // 全局变量 m
6 var m = 100
7
8 func main() {
9     n := 10
10    m := 200 // 此处声明局部变量 m
11    fmt.Println(m, n)
12 }

```

1.1.7 匿名变量

在使用多重赋值时，如果想要忽略某个值，可以使用匿名变量（anonymous variable）。匿名变量用一个下划线 `_` 表示，例如：

```

1 func getInfo() (int, string) {
2     return 10, "张三"
3 }
4
5 func main() {
6     _, username := getInfo()
7     fmt.Println(username)
8 }

```

匿名变量不占用命名空间，不会分配内存，所以匿名变量之间不存在重复声明。

⚠ Caution

1. 函数外的每个语句都必须以关键字开始（`var`、`const`、`func` 等）
2. `:=` 不能使用在函数外。
3. `_` 多用于占位，表示忽略值。

1.2 常量

相对于变量，常量是恒定不变的值，多用于定义程序运行期间不会改变的那些值。常量的声明和变量声明非常类似，只是把 `var` 换成了 `const`，常量在定义的时候必须赋值。

1.2.1 使用 `const` 定义常量

```

1 const pi = 3.1415
2 const e = 2.7182

```

多个常量也可以一起声明

```
1  const (  
2      pi = 3.1415  
3      e = 2.7182  
4  )
```

const 同时声明多个常量时， 如果省略了值则表示和上面一行的值相同。 例如：

```
1  const (  
2      n1 = 100  
3      n2 // 100  
4      n3 // 100  
5  )
```

1.3 变量、常量命名规则

1. 变量名称必须由数字、字母、下划线组成。
2. 标识符开头不能是数字。
3. 标识符不能是保留字和关键字。
4. 变量的名字是区分大小写的如：`age` 和 `Age` 是不同的变量。在实际的运用中，也建议，不要用一个单词大小写区分两个变量。
5. 变量名称一定要见名思意：变量名称建议用名词，方法名称建议用动词。
6. 变量命名一般采用驼峰式，当遇到特有名词（缩写或简称，如 DNS）的时候，特有名词根据是否私有全部大写或小写。

2. 基本数据类型

Go 语言中数据类型分为：基本数据类型和复合数据类型

- 基本数据类型有：整型、浮点型、布尔型、字符串
- 复合数据类型有：数组、切片、结构体、函数、map、通道（channel）、接口等。

2.1 整型

类型	范围	占用空间	有无符号
int8	(-128 到 127) -2^7 到 2^7-1	1 个字节	有
int16	(-32768 到 32767) -2^{15} 到 $2^{15}-1$	2 个字节	有
int32	(-2147483648 到 2147483647) -2^{31} 到 $2^{31}-1$	4 个字节	有
int64	(-9223372036854775808 到 9223372036854775807) -2^{63} 到 $2^{63}-1$	8 个字节	有
uint8	(0 到 255) 0 到 2^8-1	1 个字节	无
uint16	(0 到 65535) 0 到 $2^{16}-1$	2 个字节	无
uint32	(0 到 4294967295) 0 到 $2^{32}-1$	4 个字节	无
uint64	(0 到 18446744073709551615) 0 到 $2^{64}-1$	8 个字节	无

④ Note

实际项目中整数类型、切片、map 的元素数量等都可以用 `int` 来表示。在涉及到二进制传输、为了保持文件的结构不会受到不同编译目标平台字节长度的影响，不要使

用 `int` 和 `uint`。

```
1 | var num int64
2 | num = 123
3 | fmt.Printf("值: %d, 类型: %T", num, num)
```

`unsafe.Sizeof(n1)` 是 `unsafe` 包的一个函数，可以返回变量占用的字节数

```

1 package main
2
3 import (
4     "fmt"
5     "unsafe"
6 )
7
8 func main() {
9     var a int8 = 120
10    fmt.Printf("%T\n", a) // int8
11    fmt.Println(unsafe.Sizeof(a)) // 1
12 }

```

2.2 浮点型

Go 语言支持两种浮点型数：`float32` 和 `float64`。打印浮点数时，可以使用 `fmt` 包配合动词 `%f`，代码如下：

```

1 package main
2
3 import (
4     "fmt"
5     "math"
6 )
7
8 func main() {
9     fmt.Printf("%f\n", math.Pi) // 默认保留 6 位小数
10    fmt.Printf("%.2f\n", math.Pi) // 保留 2 位小数
11 }

```

💡 Tip

Go 语言中浮点数默认是 `float64`

2.2.1 精度丢失问题

几乎所有的编程语言都有精度丢失这个问题，这是典型的二进制浮点数精度损失问题，在定长条件下，二进制小数和十进制小数互转可能有精度丢失。

```

1 m1 := 8.2
2 m2 := 3.8
3 fmt.Println(m1 - m2) // 期望是 4.4， 结果打印出了 4.3999999999999995

```

💡 Tip

可以使用第三方包来解决精度损失问题 <https://github.com/shopspring/decimal>

2.2.2 科学计数法表示浮点类型

```
1 num1 := 1.234e3
2 num2 := 1.234e-3
3
4 fmt.Println(num1, num2)
```

2.3 布尔型

Go 语言中以 `bool` 类型进行声明布尔型数据，布尔型数据只有 `true` 和 `false` 两个值。

⚠ Caution

1. 布尔类型变量的默认值为 `false`。
2. Go 语言中不允许将整型强制转换为布尔型。
3. 布尔型无法参与数值运算，也无法与其他类型进行转换。

```
1 package main
2
3 import (
4     "fmt"
5     "unsafe"
6 )
7
8 func main() {
9     var b = true
10    fmt.Println(b, "占用字节:", unsafe.Sizeof(b)) // 占用字节: 1
11 }
```

2.4 字符串

Go 语言里的字符串的内部实现使用 UTF-8 编码。字符串的值为双引号中的内容，可以在 Go 语言的源码中直接添加非 ASCII 码字符，例如：

```
1 s1 := "hello"
2 s2 := "你好"
```

2.4.1 转义字符

Go 语言的字符串常见转义符包含回车、换行、单双引号、制表符等，如下表所示。

转义符	含义
<code>\r</code>	回车符（返回行首）
<code>\n</code>	换行符（直接跳到下一行的同列位置）
<code>\t</code>	制表符
<code>\'</code>	单引号
<code>\"</code>	双引号
<code>\\</code>	反斜杠

2.4.2 多行字符串

Go 语言中要定义一个多行字符串时，就必须使用反引号字符：

```
1 | s1 := `第一行`
2 |   第二行
3 |   第三行
4 | `
5 | fmt.Println(s1)
```

2.4.3 字符串的常用操作

求字符串的长度

```
1 | var str = "this is str"
2 | fmt.Println(len(str))
```

拼接字符串

```
1 | var str1 = "你好"
2 | var str2 = "golang"
3 | fmt.Println(str1 + str2)
4 |
5 | var str3 = fmt.Sprintf("%v %v", str1, str2)
6 | fmt.Println(str3)
```

分割字符串

```
1 | var str = "123-456-789"
2 | var arr = strings.Split(str, "-")
3 | fmt.Println(arr)
```

包含字符串


```

1 | var str = "this is golang"
2 | var flag = strings.Contains(str, "golang")
3 | fmt.Println(flag)

```

判断首字符尾字母是否包含指定字符

```

1 | var str = "this is golang"
2 | var flag = strings.HasPrefix(str, "this")
3 | fmt.Println(flag)
4 |
5 | var str = "this is golang"
6 | var flag = strings.HasSuffix(str, "go")
7 | fmt.Println(flag)

```

判断字符串出现的位置

```

1 | var str = "this is golang"
2 | var index = strings.Index(str, "is") // 从前往后
3 | fmt.Println(index)
4 |
5 | var str = "this is golang"
6 | var index = strings.LastIndex(str, "is") // 从后往前
7 | fmt.Println(index)

```

Join 拼接字符串

```

1 | var str = "123-456-789"
2 | var arr = strings.Split(str, "-")
3 | var str2 = strings.Join(arr, "*")
4 | fmt.Println(str2)

```

2.5 字符型

组成每个字符串的元素叫做字符，可以通过遍历字符串元素获得字符。字符用单引号包裹起来，如：

```

1 | a := 'a'
2 | b := '0'
3 | // 当我们直接输出 byte（字符） 的时候输出的是这个字符对应的码值
4 | fmt.Println(a)
5 | fmt.Println(b)
6 | // 如果我们要输出这个字符， 需要格式化输出
7 | fmt.Printf("%c--%c", a, b) // %c 相应 Unicode 码点所表示的字符

```

💡 Tip

字节 (byte)：是计算机中数据处理的基本单位，习惯上用大写 B 来表示，1B (byte) = 8bit (位)
一个汉子占用 3 个字节一个字母占用一个字节

```

1 a := "m"
2 fmt.Println(len(a)) // 1
3
4 b := "张"
5 fmt.Println(len(b)) // 3

```

Go 语言的字符有以下两种：

1. `byte` 或者叫 `uint8` 型，代表了 ASCII 码的一个字符。
2. `rune` 代表一个 UTF-8 字符，实际是一个 `int32`。

当需要处理中文、日文或者其他复合字符时，则需要用到 `rune` 类型。`rune` 类型实际是一个 `int32`。

```

1 s := "hello 张三"
2 for i := 0; i < len(s); i++ { // byte
3     fmt.Printf("%v(%c) ", s[i], s[i])
4 }
5 // 104(h) 101(e) 108(l) 108(l) 111(o) 32( ) 229(ǎ) 188(¼) 160( ) 228(ä) 184(,) 137(•)
6
7 for _, r := range s { // rune
8     fmt.Printf("%v(%c) ", r, r)
9 }
10 // 104(h) 101(e) 108(l) 108(l) 111(o) 32( ) 24352(张) 19977(三)

```

字符串底层是一个 `byte` 数组，所以可以和 `[]byte` 类型相互转换。字符串是不能修改的字符串是由 `byte` 字节组成，所以字符串的长度是 `byte` 字节的长度。`rune` 类型用来表示 UTF-8 字符，一个 `rune` 字符由一个或多个 `byte` 组成。

2.6 修改字符串

要修改字符串，需要先将其转换成 `[]rune` 或 `[]byte`，完成后再转换为 `string`。无论哪种转换，都会重新分配内存，并复制字节数组。

```

1 var s = "hello 张三"
2 byteStr := []byte(s)
3 byteStr[0] = 'H'
4 fmt.Println(string(byteStr)) // Hello 张三
5
6 runeStr := []rune(s)
7 runeStr[len(runeStr)-1] = '五'
8 fmt.Println(string(runeStr)) // hello 张五

```

2.7 数据类型转化

Go 语言中只有强制类型转换，没有隐式类型转换。

2.7.1 数值类型之间的相互转换

```
1 var a int8 = 20
2 var b int16 = 40
3 var c = int16(a) + b           // 要转换成相同类型才能运行
4 fmt.Printf("值: %v, 类型: %T", c, c) // 值: 60, 类型: int16
```

```
1 var d float32 = 3.2
2 var e int16 = 6
3 var f = d + float32(e)
4 fmt.Printf("值: %v, 类型: %T", f, f) // 值: 9.2, 类型: float32
```

Tip

转换的时候建议从低位转换成高位，高位转换成低位的时候如果转换不成功就会溢出，和我们想要的结果不一样

2.7.2 其他类型转换成 String 类型

使用 `fmt.Sprintf` 把其他类型转换成 `string` 类型

```
1 package main
2
3 import "fmt"
4
5 func main() {
6     var i int = 10
7     var f float32 = 3.14
8     var c byte = 'a'
9     var b bool = true
10
11     var s string
12
13     s = fmt.Sprintf("%d", i)
14     fmt.Printf("类型 %T, %v \n", s, s)
15     s = fmt.Sprintf("%f", f)
16     fmt.Printf("类型 %T, %v \n", s, s)
17     s = fmt.Sprintf("%t", b)
18     fmt.Printf("类型 %T, %v \n", s, s)
19     s = fmt.Sprintf("%c", c)
20     fmt.Printf("类型 %T, %v \n", s, s)
21 }
```

```
1 类型 string, 10
2 类型 string, 3.140000
3 类型 string, true
4 类型 string, a
```

使用 `strconv` 包里面的几种转换方法进行转换

```

1 package main
2
3 import (
4     "fmt"
5     "strconv"
6 )
7
8 func main() {
9     //1、 int 转换成 string
10    var num1 int = 20
11    s1 := strconv.Itoa(num1)
12    fmt.Printf("类型 %T, %v \n", s1, s1)
13
14    // 2、 float 转 string
15    var num2 float64 = 20.113123
16    s2 := strconv.FormatFloat(num2, 'f', 2, 64)
17    fmt.Printf("类型 %T, %v \n", s2, s2)
18
19    // 3、 bool 转 string
20    s3 := strconv.FormatBool(true)
21    fmt.Printf("类型 %T, %v \n", s3, s3)
22
23    // 4、 int64 转 string
24    var num3 int64 = 20
25    s4 := strconv.FormatInt(num3, 10)
26    fmt.Printf("类型 %T ,strs=%v \n", s4, s4)
27 }

```

2.7.3 String 类型转换成数值类型

使用 `strconv` 包里面的几种转换方法进行转换

```

1 package main
2
3 import (
4     "fmt"
5     "strconv"
6 )
7
8 func main() {
9     // 1、 string 类型转 int
10    var s1 string = "1234"
11    i, _ := strconv.ParseInt(s1, 10, 64)
12    fmt.Printf("值: %d, 类型: %T\n", i, i)
13
14    // 2、 string 转 float
15    var s2 string = "3.1415926"
16    f, _ := strconv.ParseFloat(s2, 64)
17    fmt.Printf("值: %f, 类型: %T\n", f, f)
18
19    // 3、 string 转 bool
20    var s3 string = "true"

```

```

21     b, _ := strconv.ParseBool(s3)
22     fmt.Printf("值: %t, 类型: %T\n", b, b)
23
24     // 4、string 转 byte
25     var s4 string = "abc"
26     for _, r := range s4 {
27         fmt.Printf("值: %c, 类型: %T\n", r, r)
28     }
29 }

```

⚠ Caution

在 go 语言中数值类型没法直接转换成 `bool` 类型 `bool` 类型也没法直接转换成数值类型

3. 运算符

3.1 算术运算符

```

1 // 1、+ 运算符
2 fmt.Println("10+3=", 10+3) // =13
3
4 // 2、- 运算符
5 fmt.Println("10-3=", 10-3) // =7
6
7 // 3、* 运算符
8 fmt.Println("10*3=", 10*3) // =30
9
10 // 4、/ 运算符
11 // 除法注意：如果运算的数都是整数，那么除后，去掉小数部分，保留整数部分
12 fmt.Println("10/3=", 10/3) // 3
13 fmt.Println("10.0/3=", 10.0/3) // 3.3333333333333335
14
15 // 5、% 运算符
16 // 取余注意 余数 = 被除数 - (被除数/除数) * 除数
17 fmt.Println("10%3=", 10%3) // =1
18 fmt.Println("-10%3=", -10%3) // =-1
19 fmt.Println("10%-3=", 10%-3) // =1
20 fmt.Println("-10%-3=", -10%-3) // =-1

```

⚠ Caution

`++` (自增) 和 `--` (自减) 在 Go 语言中是单独的语句，并不是运算符。

```

1  var i int = 8
2  var a int
3  a = i++ // 错误, i++ 只能独立使用
4  a = i-- // 错误, i-- 只能独立使用

```

```

1  var i int = 1
2  ++i // 错误, 在 golang 没有 前++
3  --i // 错误, 在 golang 没有 前--
4  fmt.Println("i=", i)

```

⚠ Caution

在 golang 中, ++ 和 -- 只能独立使用。在 golang 中没有前++

```

1  var i int = 1
2  i++ // 正确写法
3  fmt.Println("i=", i)

```

3.2 关系运算符

```

1  var n1 int = 9
2  var n2 int = 8
3
4  // 1、= 运算符
5  fmt.Println(n1 == n2) // false
6  // 2、!= 运算符
7  fmt.Println(n1 != n2) // true
8  // 3、> 运算符
9  fmt.Println(n1 > n2) // true
10 // 4、≥ 运算符
11 fmt.Println(n1 ≥ n2) // true
12 // 5、< 运算符
13 fmt.Println(n1 < n2) // false
14 // 6、≤ 运算符
15 fmt.Println(n1 ≤ n2) // false

```

3.3 逻辑运算符

```

1  // 1、&& 逻辑运算符
2  var age int = 40
3  if age > 30 && age < 50 {
4      fmt.Println("ok1")
5  }
6  if age > 30 && age < 40 {
7      fmt.Println("ok2")
8  }
9

```

```

10 // 2、|| 逻辑运算符
11 if age > 30 || age < 50 {
12     fmt.Println("ok3")
13 }
14 if age > 30 || age < 40 {
15     fmt.Println("ok4")
16 }
17 // 3、! 逻辑运算符
18 if age > 30 {
19     fmt.Println("ok5")
20 }
21 if !(age > 30) {
22     fmt.Println("ok6")
23 }

```

逻辑运算符短路：在逻辑运算中，当能够确定整个表达式的结果时，就不再计算剩余部分的特性。

```

1 // 示例1: && 短路
2 fmt.Println("=== && 短路演示 ===")
3
4 a, b := 10, 20
5
6 // 第一个条件为 false，第二个条件不会执行
7 if a > 100 && b > 0 {
8     fmt.Println("条件成立")
9 } else {
10     fmt.Println("条件不成立，b>0不会被执行")
11 }
12
13 // 示例2: || 短路
14 fmt.Println("\n=== || 短路演示 ===")
15
16 // 第一个条件为true，第二个条件不会执行
17 if a < 100 || b < 0 {
18     fmt.Println("条件成立，b<0不会被执行")
19 }

```

4. 流程控制

4.1 if else

```

1 var score int = 80
2
3 if score ≥ 90 {
4     fmt.Println("优秀")
5 } else if score ≥ 80 {
6     fmt.Println("良好")
7 } else if score ≥ 60 {
8     fmt.Println("及格")

```

```
9   } else {
10       fmt.Println("不及格")
11   }
```

⚠ Caution

Go 语言规定与 `if` 匹配的左括号 `{` 必须与 `if` 和表达式放在同一行，`{` 放在其他位置会触发编译错误。同理，与 `else` 匹配的 `{` 也必须与 `else` 写在同一行，`else` 也必须与上一个 `if` 或 `else if` 右边的大括号在同一行。

⚠ Caution

Go 语言中没有三目运算符

4.2 for

Go 语言中的所有循环类型均可以使用 `for` 关键字来完成。

```
1   for i := 0; i < 10; i++ {
2       fmt.Println(i)
3   }
```

`for` 循环的初始语句可以被忽略，但是初始语句后的分号必须要写，例如：

```
1   i := 0
2   for ; i < 10; i++ {
3       fmt.Println(i)
4   }
```

`for` 循环的初始语句和结束语句都可以省略，例如：

```
1   i := 0
2   for i < 10 {
3       fmt.Println(i)
4       i++
5   }
```

⚠ Caution

Go 语言中是没有 `while` 语句的，我们可以通过 `for` 代替

`for` 循环可以通过 `break`、`goto`、`return`、`panic` 语句强制退出循环。


```

1  k := 1
2  for {
3      if k ≤ 10 {
4          fmt.Println(k)
5      } else {
6          break // break 就是跳出这个 for 循环
7      }
8      k++
9  }

```

4.3 for range

Go 语言中可以使用 `for range` 遍历数组、切片、字符串、map 及通道 (channel)。通过 `range` 遍历的返回值有以下规律：

1. 数组、切片、字符串返回索引和值。
2. map 返回键和值
3. 通道 (channel) 只返回通道内的值。

```

1  s := "hello world"
2  for index, value := range s {
3      fmt.Printf("%d:%c\n", index, value)
4  }
5
6  slice := []int{1, 2, 3, 4, 5}
7  for index, value := range slice {
8      fmt.Printf("%d:%d\n", index, value)
9  }

```

4.4 switch case

使用 `switch` 语句可方便地对大量的值进行条件判断。Go 语言规定每个 `switch` 只能有一个 `default` 分支。

```

1  suffix := ".a"
2  switch suffix {
3      case ".html":
4          fmt.Println("text/html")
5      case ".css":
6          fmt.Println("text/css")
7      case ".js":
8          fmt.Println("text/javascript")
9      default:
10         fmt.Println("格式错误")
11

```

Go 语言中每个 `case` 语句中可以不加 `break`，不加 `break` 也不会出现穿透的现象。一个分支可以有多个值，多个 `case` 值中间使用英文逗号分隔。

```

1  n := 2
2  switch n {
3      case 1, 3, 5, 7, 9:
4          fmt.Println("奇数")
5      case 2, 4, 6, 8:
6          fmt.Println("偶数")
7      default:
8          fmt.Println(n)
9  }

```

`fallthrough` 是 Go 语言中 `switch` 语句的一个特殊关键字，用于强制执行下一个 `case` 分支，而不管下一个 `case` 的条件是否满足。

```

1  num := 2
2
3  switch num {
4      case 1:
5          fmt.Println("数字是 1")
6      case 2:
7          fmt.Println("数字是 2")
8      case 3:
9          fmt.Println("数字是 3")
10     default:
11         fmt.Println("其他数字")
12 }
13
14 // 输出:
15 // 数字是 2

```

```

1  num := 2
2
3  switch num {
4      case 1:
5          fmt.Println("数字是 1")
6      case 2:
7          fmt.Println("数字是 2")
8          fallthrough // 强制执行下一个 case
9      case 3:
10         fmt.Println("数字是 3")
11     case 4:
12         fmt.Println("数字是 4")
13     default:
14         fmt.Println("其他数字")
15 }
16
17 // 输出:
18 // 数字是 2
19 // 数字是 3

```

4.5 break

Go 语言中 `break` 语句用于以下几个方面：

1. 用于循环语句中跳出循环，并开始执行循环之后的语句。
2. `break` 在 `switch`（开关语句）中在执行一条 `case` 后跳出语句的作用。
3. 在多重循环中，可以用标号 `label` 标出想 `break` 的循环。

```
1  label:
2      for i := 0; i < 2; i++ {
3          for j := 0; j < 10; j++ {
4              if j == 2 {
5                  break label
6              }
7              fmt.Println("i, j", i, j)
8          }
9      }
```

4.6 continue

`continue` 语句可以结束当前循环，开始下一次的循环迭代过程，仅限在 `for` 循环内使用。

💡 Tip

在 `continue` 语句后添加标签时，表示开始标签对应的循环。

```
1  here:
2      for i := 0; i < 2; i++ {
3          for j := 0; j < 4; j++ {
4              if j == 2 {
5                  continue here
6              }
7              fmt.Println("i, j", i, j)
8          }
9      }
```

4.7 goto

`goto` 语句通过标签进行代码间的无条件跳转。`goto` 语句可以在快速跳出循环、避免重复退出上有一定的帮助。

```

1   for i := 0; i < 10; i++ {
2       for j := 0; j < 10; j++ {
3           if j == 2 {
4               goto breakTag
5           }
6           fmt.Println("i, j", i, j)
7       }
8   }
9   return
10  breakTag:
11      fmt.Println("breakTag")

```

5. 数组

数组是指一系列同一类型数据的集合。数组中包含的每个数据被称为数组元素，这种类型可以是任意的原始类型，比如 `int`、`string` 等，也可以是用户自定义的类型。

一个数组包含的元素个数被称为数组的长度。

在 `GoLang` 中数组是一个长度固定的数据类型，数组的长度是类型的一部分，也就是说 `[5]int` 和 `[10]int` 是两个不同的类型。

5.1 数组的声明

```

1   var a [3]int
2   var b [4]int

```

数组的长度必须是常量，并且长度是数组类型的一部分。一旦定义，**长度不能变**。数组可以通过下标进行访问，下标是从 `0` 开始，最后一个元素下标是 `len-1`

5.2 数组的初始化

初始化数组时可以使用初始化列表来设置数组元素的值。

```

1   var testArray [5]int
2   var numArray = [3]int{1, 2, 3}
3   var cityArray = [3]string{"北京", "上海", "广州"}
4
5   fmt.Println(testArray) // [0 0 0 0 0]
6   fmt.Println(numArray) // [1 2 3]
7   fmt.Println(cityArray) // [北京 上海 广州]

```

除此之外，还可以让编译器根据初始值的个数自行推断数组的长度。

```

1 var testArray = [...]int{}
2 var numArray = [...]int{1, 2, 3}
3 var cityArray = [...]string{"北京", "上海", "广州"}
4
5 fmt.Println(testArray)
6 fmt.Println(numArray)
7 fmt.Println(cityArray)

```

我们还可以使用指定索引值的方式来初始化数组。

```

1 a := [...]int{1: 1, 3: 5}
2 fmt.Println(a) // [0 1 0 5]
3 fmt.Printf("type of a:%T\n", a) // type of a:[4]int

```

5.3 数组的遍历

```

1 // 1. for 循环遍历
2 for i := 0; i < len(array); i++ {
3     fmt.Println(array[i])
4 }
5
6 // 2. for range 遍历
7 for _, v := range array {
8     fmt.Println(v)
9 }

```

5.4 数组是值类型

数组是值类型，赋值和传参会复制整个数组。因此改变副本的值，不会改变本身的值。

```

1 func modifyArray(x [3]int) {
2     x[0] = 100
3 }
4
5 func modifyArray2(x [3][2]int) {
6     x[2][0] = 100
7 }
8
9 func main() {
10    a := [3]int{10, 20, 30}
11    modifyArray(a) // 在 modify 中修改的是 a 的副本 x
12    fmt.Println(a) // [10 20 30]
13
14    b := [3][2]int{
15        {1, 1},
16        {1, 1},
17        {1, 1},
18    }
19    modifyArray2(b) // 在 modify 中修改的是 b 的副本 x

```

```
20     fmt.Println(b) // [[1 1] [1 1] [1 1]]
21 }
```

① Note

1. 数组支持 `==`、`≠` 操作符，因为内存总是被初始化过的。
2. `[n]*T` 表示指针数组，`*[n]T` 表示数组指针。

6. 切片

切片（Slice）是一个拥有相同类型元素的可变长度的序列。它是基于数组类型做的一层封装。它非常灵活，支持自动扩容。

切片是一个引用类型，它的内部结构包含地址、长度和容量。

6.1 切片的声明

```
1  var a []string
2  var b []int = []int{1, 2, 3}
3
4  fmt.Println(a == nil) // true
5  fmt.Println(b == nil) // false
```

① Note

当你声明了一个切片，却没有给其赋值，此时该切片为默认值 `nil`

📢 Important

当你声明了一个变量，但却还没有赋值时，Golang 中会自动给你的变量赋值一个默认零值。

类型类别	具体类型	零值（默认值）
布尔型	<code>bool</code>	<code>false</code>
整型	<code>int</code> , <code>int8</code> , <code>int16</code> , <code>int32</code> , <code>int64</code> , <code>uint</code> , <code>uint8</code> , <code>uint16</code> , <code>uint32</code> , <code>uint64</code>	<code>0</code>
浮点型	<code>float32</code> , <code>float64</code>	<code>0.0</code>
复数型	<code>complex64</code> , <code>complex128</code>	<code>(0+0i)</code>
字符串	<code>string</code>	<code>""</code>
数组	<code>[n]T</code>	<code>[n]T{}</code>
切片	<code>[]T</code>	<code>nil</code>
映射	<code>map[K]V</code>	<code>nil</code>
通道	<code>chan T</code>	<code>nil</code>
指针	<code>*T</code>	<code>nil</code>
函数	<code>func(...)</code>	<code>nil</code>
接口	<code>interface{}</code>	<code>nil</code>
结构体	<code>struct{}</code>	各字段的零值
字节类型	<code>byte</code> (<code>uint8</code> 别名)	<code>0</code>
符文类型	<code>rune</code> (<code>int32</code> 别名)	<code>0</code>
错误类型	<code>error</code>	<code>nil</code>

6.2 切片的遍历

切片的遍历与数组类似，可以使用 `for` 或者 `for range` 遍历。

```

1  var strSlice []string = []string{"hello", "world", "golang"}
2
3  // 1. 使用 for 遍历切片
4  for i := 0; i < len(strSlice); i++ {
5      fmt.Println(strSlice[i])
6  }
7
8  // 2. 使用 for range 遍历切片
9  for _, v := range strSlice {
10     fmt.Println(v)
11 }

```

6.3 基于数组定义切片

由于切片的底层就是一个数组，所以我们可以基于数组定义切片。

```

1  numArray := [5]int{1, 2, 3, 4, 5}
2  numSlice := numArray[1:3] // 基于数组创建切片
3  fmt.Println(numSlice) // [2 3]

```

6.4 切片的长度和容量

切片拥有自己的长度和容量，我们可以通过使用内置的 `len()` 函数求长度，使用内置的 `cap()` 函数求切片的容量。

- 切片的长度就是它所包含的元素个数。
- 切片的容量是从它的第一个元素开始数，到其底层数组元素末尾的个数。

```

1  s := []int{2, 3, 5, 7, 11, 13}
2  fmt.Println(s)
3  fmt.Printf("长度: %v, 容量: %v\n", len(s), cap(s))
4
5  c := s[:2]
6  fmt.Println(c)
7  fmt.Printf("长度: %v, 容量: %v\n", len(c), cap(c))
8
9  d := s[1:3]
10 fmt.Println(d)
11 fmt.Printf("长度: %v, 容量: %v\n", len(d), cap(d))
12
13 // 输出:
14 // [2 3 5 7 11 13]
15 // 长度: 6, 容量: 6
16 // [2 3]
17 // 长度: 2, 容量: 6
18 // [3 5]
19 // 长度: 2, 容量: 5

```


6.5 使用 `make()` 函数声明

如果需要动态的创建一个切片，我们就需要使用内置的 `make()` 函数。

```
1 | make([]T, size, cap)
```

其中：

- `[]T`：切片的元素类型
- `size`：切片中的元素数量
- `cap`：切片的容量

```
1 | var a = make([]int, 2, 10)
2 | fmt.Println(a) // [0 0]
3 | fmt.Println(len(a)) // 2
4 | fmt.Println(cap(a)) // 10
```

⚠ Caution

切片之间是不能比较的，我们不能使用 `==` 操作符来判断两个切片是否含有全部相等元素。切片唯一合法的比较操作是和 `nil` 比较。

6.6 切片的基础操作

6.6.1 `append()` 方法

当底层数组不能容纳新增的元素时，切片就会自动按照一定的策略进行扩容，此时该切片指向的底层数组就会更换。扩容操作往往发生在 `append()` 函数调用时，所以我们通常都需要用原变量接收 `append()` 函数的返回值。

错误写法：

```
1 | s := []int{1, 2, 3, 5, 6, 7}
2 | s[6] = 8
3 | fmt.Println(s) //index out of range [6] with length 6
```

正确写法：

```
1 | var numSlice []int
2 | for i := 0; i < 10; i++ {
3 |     numSlice = append(numSlice, i)
4 | }
```

1. `append()` 函数将元素追加到切片的最后并返回该切片。
2. 切片 `numSlice` 的容量按照 1, 2, 4, 8, 16 这样的规则自动进行扩容，每次扩容后都是扩容前的 2 倍。

```

1  var citySlice []string
2  // 添加一个元素
3  citySlice = append(citySlice, "北京")
4
5  // 添加多个元素
6  citySlice = append(citySlice, "上海", "广州", "深圳")
7
8  // 添加切片
9  citySlice = append(citySlice, []string{"杭州", "苏州"}...)
10 fmt.Println(citySlice) // [北京 上海 广州 深圳 杭州 苏州]

```

6.6.2 copy() 方法

```

1  a := []int{1, 2, 3, 4, 5}
2  b := a
3  fmt.Println(a) // [1 2 3 4 5]
4  fmt.Println(b) // [1 2 3 4 5]
5  b[0] = 1000
6  fmt.Println(a) // [1000 2 3 4 5]
7  fmt.Println(b) // [1000 2 3 4 5]

```

由于切片是引用类型，所以在修改 `b` 的同时 `a` 的值也会发生变化。Go 语言内建的 `copy()` 函数可以迅速地将一个切片的数据复制到另外一个切片空间中，`copy()` 函数的使用格式如下：

```

1  copy(destSlice, srcSlice []T)

```

其中：

- `destSlice`：目标切片
- `srcSlice`：数据来源切片

```

1  s1 := []int{1, 2, 3, 4, 5}
2  s2 := make([]int, 5)
3
4  copy(s2, s1) // 将 s1 拷贝到 s2
5  fmt.Println(s1) // [1 2 3 4 5]
6  fmt.Println(s2) // [1 2 3 4 5]
7
8  s2[0] = 100
9  fmt.Println(s1) // [1 2 3 4 5]
10 fmt.Println(s2) // [100 2 3 4 5]

```

6.6.3 删除元素

Go 语言中并没有删除切片元素的专用方法，我们可以使用切片本身的特性来删除元素。要从切片 `a` 中删除索引为 `index` 的元素，操作方法是

```

1  a = append(a[:index], a[index+1:]...)

```

```

1 func popString(srcSlice []string, index int) []string {
2     // 删除切片中的元素
3     return append(srcSlice[:index], srcSlice[index+1:]...)
4 }

```

6.7 切片排序

6.7.1 冒泡排序

核心理想：**相邻比较和交换**，就像水中气泡上浮一样，它通过反复比较相邻元素，将较大的元素逐步向后移动，每一轮都会将当前未排序部分的最大值“冒泡”到正确位置，直到整个序列有序。

```

1 func bubbleSort(numSlice []int) []int {
2     // 边界条件
3     if len(numSlice) ≤ 1 {
4         return numSlice
5     }
6
7     // 冒泡排序
8     for i := 0; i < len(numSlice)-1; i++ {
9         // 第 i 趟冒泡
10        isSwap := false
11        for j := 0; j < len(numSlice)-i-1; j++ {
12            if numSlice[j] > numSlice[j+1] {
13                numSlice[j], numSlice[j+1] = numSlice[j+1], numSlice[j]
14                isSwap = true
15            }
16        }
17        if !isSwap {
18            break
19        }
20    }
21    return numSlice
22 }

```

6.7.2 选择排序

核心理想：**选取最小元素**，它把序列分为已排序和未排序两部分，每次从未排序部分中选出最小的元素，直接将其放到已排序部分的末尾，如同打擂台选冠军一样逐个确定位置，直到所有元素归位。

```

1 func selecSort(numSlice []int) []int {
2     // 边界条件
3     if len(numSlice) ≤ 1 {
4         return numSlice
5     }
6
7     for i := 0; i < len(numSlice)-1; i++ {
8         // 第 i 趟选择
9         minIndex := i

```

```

10     // 找到最小值的索引
11     for j := i + 1; j < len(numSlice); j++ {
12         if numSlice[j] < numSlice[minIndex] {
13             minIndex = j
14         }
15     }
16     if minIndex != i {
17         // 将未排序区间最小值交换到已排序区间的末尾
18         numSlice[i], numSlice[minIndex] = numSlice[minIndex], numSlice[i]
19     }
20 }
21
22 return numSlice
23 }

```

6.7.3 插入排序

核心理念：**构建有序序列**，就像整理手中的扑克牌，它将序列视为已排序和未排序两部分，每次从未排序部分取出第一个元素，在已排序部分中从后向前扫描，找到合适位置插入，从而逐步扩大有序区间，直到全部有序。

```

1  func insertSort(numSlice []int) []int {
2      // 边界条件
3      if len(numSlice) ≤ 1 {
4          return numSlice
5      }
6
7      // 遍历无序区间，从第二个元素开始遍历
8      for i := 1; i < len(numSlice); i++ {
9          // 保存当前元素
10         current := numSlice[i]
11         // 找到当前元素应该插入的位置
12         j := i - 1
13
14         // 如果 j ≥ 0 且 numSlice[j] > current，
15         // 则说明当前元素在有序区间中，则将当前元素插入有序区间中
16         for j ≥ 0 && numSlice[j] > current {
17             numSlice[j+1] = numSlice[j]
18             j--
19         }
20         numSlice[j+1] = current
21     }
22     return numSlice
23
24 }

```

6.7.4 sort 包

对于 `int`、`float64` 和 `string` 数组或是切片的排序，`go` 分别提供了 `sort.Ints()`、`sort.Float64s()` 和 `sort.Strings()` 函数，默认都是从小到大排序。

```
1  intList := []int{2, 4, 3, 5, 7, 6, 9, 8, 1, 0}
2  floatList := []float64{4.2, 5.9, 12.4, 10.2, 50.7, 99.9, 31.4, 27.81828, 3.14}
3  stringList := []string{"a", "c", "b", "z", "x", "w", "y", "d", "f", "i"}
4
5  sort.Ints(intList)
6  sort.Float64s(floatList)
7  sort.Strings(stringList)
8  fmt.Println(intList) // [0 1 2 3 4 5 6 7 8 9]
9  fmt.Println(floatList) // [3.14 4.2 5.9 10.2 12.4 27.81828 31.4 50.7 99.9]
10 fmt.Println(stringList) // [a b c d f i w x y z]
```

Golang 的 `sort` 包可以使用 `sort.Reverse(slice)` 来调换 `slice.Interface.Less`，所以逆序排序函数可以这么写。

```
1  sort.Sort(sort.Reverse(sort.IntSlice(intList)))
2  fmt.Println(intList) // [9 8 7 6 5 4 3 2 1 0]
```

7. 哈希表

7.1 哈希表的声明

Go 语言的哈希表可以通过 `map` 实现，`map` 是一种无序的基于 `key-value` 的数据结构，Go 语言中的 `map` 是引用类型，必须初始化才能使用。

```
1  map[KeyType]ValueType
```

其中：

- `KeyType`：表示键的类型。
- `ValueType`：表示键对应的值的类型。

① Note

`map` 类型的变量默认初始值为 `nil`，需要使用 `make()` 函数来分配内存。

```
1  make(map[KeyType]ValueType, [cap])
```

① Caution

获取 `map` 的容量不能使用 `cap`，`cap` 返回的是数组切片分配的空间大小，根本不能用于 `map`。要获取 `map` 的容量，可以用 `len()` 函数。

`map` 中的数据都是成对出现的，`map` 的基本使用示例代码如下：

```
1 hashMap := make(map[string]int)
2 hashMap["a"] = 1
3 hashMap["b"] = 2
4 hashMap["c"] = 3
5 fmt.Println(hashMap) // map[a:1 b:2 c:3]
6 fmt.Println(hashMap["a"]) // 1
```

`map` 也支持在声明的时候填充元素，例如：

```
1 userInfo := map[string]string{
2     "username": "zhangsan",
3     "password": "123456",
4 }
5 fmt.Println(userInfo) // map[password:123456 username:zhangsan]
```

7.2 判断键是否存在

Go 语言中有个判断 `map` 中键是否存在的特殊写法，格式如下：

```
1 value, ok := map[key]
```

```
1 scoreMap := make(map[string]int)
2 scoreMap["张三"] = 90
3 scoreMap["李四"] = 80
4
5 // 如果 key 存在 ok 为 true,v 为对应的值；
6 // 不存在 ok 为 false,v 为值类型的零值
7 v, ok := scoreMap["张三"]
8 if ok {
9     fmt.Println(v) // 90
10 } else {
11     fmt.Println("没有该用户")
12 }
```

7.3 哈希表的遍历

Go 语言中使用 `for range` 遍历 `map`。

```

1 | scoreMap := make(map[string]int)
2 | scoreMap["张三"] = 90
3 | scoreMap["李四"] = 80
4 | scoreMap["王五"] = 70
5 |
6 | for k, v := range scoreMap {
7 |     fmt.Println(k, v)
8 | }

```

我们只想遍历 `key` 的时候，可以按下面的写法：

```

1 | for k := range scoreMap {
2 |     fmt.Println(k)
3 | }

```

⚠ Caution

遍历 `map` 时的元素顺序与添加键值对的顺序无关。

7.4 delete() 方法

使用 `delete()` 内建函数从 `map` 中删除一组键值对，`delete()` 函数的格式如下：

```

1 | delete(map, key)

```

其中：

- `map`：表示要删除键值对的 `map` 对象
- `key`：表示要删除的键值对的键

```

1 | scoreMap := make(map[string]int)
2 | scoreMap["张三"] = 90
3 | scoreMap["李四"] = 80
4 | scoreMap["王五"] = 70
5 | fmt.Println(scoreMap) // map[张三:90 李四:80 王五:70]
6 |
7 | // delete 方法删除
8 | delete(scoreMap, "张三")
9 | fmt.Println(scoreMap) // map[李四:80 王五:70]

```

8. 函数

函数是组织好的、可重复使用的、用于执行指定任务的代码块。Go 语言中支持：函数、匿名函数和闭包。

8.1 函数的声明

Go 语言中定义函数使用 `func` 关键字，具体格式如下：

```
1 func funcName (para Type) (return Type) {  
2     funcBody  
3 }
```

其中：

- `funcName`：函数名，由字母、数字、下划线组成。但函数名的第一个字母不能是数字。在同一个包内，函数名也称不能重名。
- `para Type`：参数，由参数变量和参数变量的类型组成，多个参数之间使用 `,` 分隔。
- `return Type`：返回值，由返回值变量和其变量类型组成，也可以只写返回值的类型，多个返回值必须用 `()` 包裹，并用 `,` 分隔。
- `funcBody`：函数体，实现指定功能的代码块。

```
1 func intSum(x int, y int) int {  
2     return x + y  
3 }
```

8.2 函数的调用

定义了函数之后， 我们可以通过 `funcName()` 的方式调用函数。 例如我们调用上面定义函数，代码如下：

```
1 func main() {  
2     res := intSum(10, 20)  
3     fmt.Println(res)  
4 }
```

Tip

调用有返回值的函数时，可以不接收其返回值。

8.3 函数参数

8.3.1 类型简写

函数的参数中如果相邻变量的类型相同，则可以省略类型，例如：

```
1 func intSum(x, y int) int {  
2     return x + y  
3 }
```


8.3.2 可变参数

可变参数是指函数的参数数量不固定。Go 语言中的可变参数通过在参数名后加 `...` 来标识。

Tip

可变参数通常要作为函数的最后一个参数。

```
1 func intSum2(x ...int) int {
2     fmt.Printf("%T\n", x) // []int
3     sum := 0
4     for _, v := range x {
5         sum = sum + v
6     }
7     return sum
8 }
```

8.4 函数返回值

Go 语言中通过 `return` 关键字向外输出返回值。

8.4.1 函数多返回值

Go 语言中函数支持多返回值，函数如果有多个返回值时必须用 `()` 将所有返回值包裹起来。

```
1 func calc(x, y int) (int, int) {
2     sum := x + y
3     sub := x - y
4     return sum, sub
5 }
```

8.4.2 返回值命名

函数定义时可以给返回值命名，并在函数体中直接使用这些变量，最后通过 `return` 关键字返回。

```
1 func calc(x, y int) (sum, sub int) {
2     sum = x + y
3     sub = x - y
4     return
5 }
```

8.5 函数类型与变量

我们可以使用 `type` 关键字来定义一个函数类型，具体格式如下：

```
1 type Calculation func(int, int) int
```

凡是满足这个条件的函数都是 `Calculation` 类型的函数，例如下面的 `add()` 和 `sub()` 是 `Calculation` 类型。

```
1 | var c Calculation = add
2 | fmt.Printf("%T\n", c) // main.Calculation
3 | fmt.Println(c(1, 2)) // 3
```

8.6 高级函数

8.6.1 函数作为参数

```
1 | func add(x, y int) int {
2 |     return x + y
3 | }
4 |
5 | func calc(x, y int, operator func(int, int) int) int {
6 |     return operator(x, y)
7 | }
8 |
9 | func main() {
10 |     ret2 := calc(10, 20, add)
11 |     fmt.Println(ret2) // 30
12 | }
```

8.6.2 函数作为返回值

```
1 | func doCalc(s string) func(int, int) int {
2 |     switch s {
3 |     case "+":
4 |         return add
5 |     case "-":
6 |         return sub
7 |     default:
8 |         return nil
9 |     }
10 | }
11 |
12 | func main() {
13 |     var a = doCalc("+")
14 |     fmt.Println(a(5, 2)) // 7
15 | }
```

8.7 匿名函数和闭包

8.7.1 匿名函数

在 Go 语言中函数内部不能再像之前那样定义函数了，只能定义匿名函数。匿名函数就是没有函数名的函数，匿名函数的定义格式如下：

```
1 func(para Type) (return Type) {
2     funcBody
3 }
```

匿名函数因为没有函数名，所以没办法像普通函数那样调用，所以匿名函数需要保存到某个变量或者作为立即执行函数：

```
1 fmt.Println(
2     func(x, y int) int {
3         return x + y
4     }(10, 20)) // 30
```

💡 Tip

匿名函数多用于实现回调函数和闭包

8.7.2 闭包

闭包可以理解成**定义在一个函数内部的函数**。在本质上，闭包是将函数内部和函数外部连接起来的桥梁。或者说是函数和其引用环境的组合体。首先我们来看一个例子：

```
1 package main
2
3 import "fmt"
4
5 func adder() func(int) int {
6     var x int
7
8     return func(y int) int {
9         x += y
10        return x
11    }
12 }
13
14 func main() {
15     var f = adder()
16     fmt.Println(f(5)) // 5
17     fmt.Println(f(5)) // 10
18     fmt.Println(f(5)) // 15
19 }
20
```

变量 `f` 是一个函数并且它引用了其外部作用域中的 `x` 变量，此时 `f` 就是一个闭包。在 `f` 的生命周期内，变量 `x` 也一直有效。

```
1 func Calc(base int) (func(int) int, func(int) int) {
2     add := func(y int) int {
3         base += y
4         return base
5     }
6     sub := func(y int) int {
7         base -= y
8         return base
9     }
10    return add, sub
11 }
12
13 func main() {
14     f1, f2 := Calc(10)
15     fmt.Println(f1(5), f2(5)) // 15 10
16     fmt.Println(f1(5), f2(5)) // 15 10
17     fmt.Println(f1(5), f2(5)) // 15 10
18 }
```

8.8 defer

Go 语言中的 `defer` 语句会将其后面跟随的语句进行延迟处理。在 `defer` 归属的函数即将返回时，将延迟处理的语句按 `defer` 定义的逆序进行执行，也就是说，先被 `defer` 的语句最后被执行，最后被 `defer` 的语句，最先被执行。

```
1 func main() {
2     fmt.Println("Start")
3     defer fmt.Println(1)
4     defer fmt.Println(2)
5     defer fmt.Println(3)
6     fmt.Println("End")
7 }
8
9 // 输出：
10 // Start
11 // End
12 // 3
13 // 2
14 // 1
```

💡 Tip

由于 `defer` 语句延迟调用的特性，所以 `defer` 语句能非常方便的处理资源释放问题。比如：资源清理、文件关闭、解锁及记录时间等

8.8.1 defer 执行时机

在 Go 语言的函数中 `return` 语句在底层并不是原子操作，它分为给返回值赋值和 `RET` 指令两步。而 `defer` 语句执行的时机就在返回值赋值操作后，`RET` 指令执行前。

```
1 func f1() int {
2     x := 5
3     defer func() { x = 10 }()
4     return x
5     // 返回 5，不是 10
6 }
```

底层执行顺序：

1. 创建匿名返回值 `r`（在栈上）
2. 将 `x` 的值 5 赋值给 `r`
3. 执行 `defer: x = 10`（不影响 `r`）
4. 返回 `r` 的值 5

8.8.2 defer 示例

```
1 func calc(index string, a, b int) int {
2     res := a + b
3     fmt.Println(index, a, b, res)
4     return res
5 }
6
7 func main() {
8     x := 1
9     y := 2
10    defer calc("AA", x, calc("A", x, y))
11    x = 10
12    defer calc("BB", x, calc("B", x, y))
13    y = 20
14 }
15
16 // 输出：
17 // A 1 2 3
18 // B 10 2 12
19 // BB 10 12 22
20 // AA 1 3 4
```

💡 Tip

`defer` 注册要延迟执行的函数时该函数所有的参数都需要确定其值。

8.9 panic()/recover()

Go 语言使用 `panic()/recover()` 模式来处理错误。`panic()` 可以在任何地方引发，但 `recover()` 只有在 `defer` 调用的函数中有效。

8.9.1 panic()/recover() 的基本使用

```
1 package main
2
3 import "fmt"
4
5 func f1() {
6     fmt.Println("hello")
7 }
8
9 func f2() {
10     defer func() {
11         if err := recover(); err != nil {
12             fmt.Println("recover:", err)
13         }
14     }()
15     panic("panic")
16 }
17
18 func main() {
19     f1()
20     f2()
21     fmt.Println("world")
22 }
23
24 // 输出:
25 // hello
26 // recover: panic
27 // world
```

⚠ Caution

1. `recover()` 必须搭配 `defer` 使用。
2. `defer` 一定要在可能引发 `panic()` 的语句之前定义。

8.9.2 defer/recover() 实现异常处理

```
1 func fn2() {
2     defer func() {
3         err := recover()
4         if err != nil {
5             fmt.Println(err) // 抛出除 0 异常
6         }
7     }()
8
9     num1 := 10
10    num2 := 0
11    res := num1 / num2
12    fmt.Println("res=", res)
13 }
```

8.9.3 defer/panic()/recover() 抛出异常

```
1 func readFile(fileName string) error {
2     if fileName == "main.go" {
3         return nil
4     }
5     return errors.New("读取文件错误")
6 }
7
8 func f3() {
9     defer func() {
10        err := recover()
11        if err != nil {
12            fmt.Println(err)
13        }
14    }()
15    err := readFile("xxx.go")
16    if err != nil {
17        panic(err)
18    }
19 }
20
21 func main() {
22     f3()
23     fmt.Println("continue...")
24 }
25
26 // 输出:
27 // 读取文件错误
28 // continue...
```

9. 指针

指针也是一个变量，但它是一种特殊的变量，它存储的数据不是一个普通的值，而是另一个变量的内存地址。

Go 语言中的指针操作非常简单，我们只需要记住两个符号：`&`（取地址）和 `*`（根据地址取值）

9.1 指针地址和指针类型

每个变量在运行时都拥有一个地址，这个地址代表变量在内存中的位置。Go 语言中使用 `&` 字符放在变量前面对变量进行取地址操作。Go 语言中的值类型（`int`、`float`、`bool`、`string`、`array`、`struct`）都有对应的指针类型，如：`*int`、`*int64`、`*string` 等。

```
1 | ptr := &v // 比如 v 的类型为 T
```

其中：

- `v`：代表被取地址的变量，类型为 `T`。
- `ptr`：用于接收地址的变量，`ptr` 的类型就为 `*T`，称做 `T` 的指针类型。`*` 代表指针。

```
1 | func main() {
2 |     var a = 10
3 |     var b = &a
4 |     fmt.Printf("%v, %T\n", a, a)
5 |     fmt.Printf("%v, %T\n", b, b)
6 | }
7 |
8 | // 输出：
9 | // 10, int
10 | // 0xc00000a0b8, *int
```

9.2 指针取值

在对普通变量使用 `&` 操作符取地址后会获得这个变量的指针，然后可以对指针使用 `*` 操作，也就是指针取值，代码如下。

```
1 | func main() {
2 |     var a = 10
3 |     var b = &a
4 |
5 |     c := *b // 指针取值（根据指针的值去内存取值）
6 |     fmt.Printf("type of c:%T\n", c)
7 |     fmt.Printf("value of c:%v\n", c)
8 | }
9 |
10 | // 输出：
11 | // type of c:int
12 | // value of c:10
```


取地址操作符 `&` 和取值操作符 `*` 是一对互补操作符，`&` 取出地址，`*` 根据地址取出地址指向的值。

```
1 func modify1(x int) {
2     x = 100
3 }
4
5 func modify2(x *int) {
6     *x = 100
7 }
8
9 func main() {
10    a := 10
11    modify1(a)
12    fmt.Printf("%d\n", a) // 10
13    modify2(&a)
14    fmt.Printf("%d\n", a) // 100
15 }
```

9.3 new()/make()

在 Go 语言中对于引用类型的变量，在使用时不仅要声明它，还要为它分配内存空间。而对于值类型的声明不需要分配内存空间，是因为它们在声明的时候已经默认分配好了内存空间。Go 语言中 `new()` 和 `make()` 是内建的两个函数，主要用来分配内存。

9.3.1 new() 函数分配内存

`new()` 是一个内置的函数，它的函数签名如下：

```
1 func new(Type) *Type
```

其中：

- `Type` 表示类型，`new()` 函数只接受一个参数，这个参数是一个类型
- `*Type` 表示类型指针，`new()` 函数返回一个指向该类型内存地址的指针。

实际开发中 `new()` 函数不太常用，使用 `new()` 函数得到的是一个类型的指针，并且该指针对应的值为该类型的零值。

```
1 a := new(int)
2 b := new(bool)
3
4 fmt.Printf("%v, %T\n", a, a) // 0xc00009e068, *int
5 fmt.Printf("%v, %T\n", b, b) // 0xc00009e080, *bool
6 fmt.Printf("%v, %T\n", *a, *a) // 0, int
7 fmt.Printf("%v, %T\n", *b, *b) // false, bool
```

```
1 | var a *int
2 | a = new(int)
3 | *a = 10
4 | fmt.Println(*a)
```

9.3.2 make() 函数分配内存

`make()` 也是用于内存分配的，区别于 `new()`，它只用于 `slice`、`map` 以及 `channel` 的内存创建，而且它返回的类型就是这三个类型本身，而不是他们的指针类型，因为这三种类型就是引用类型，所以就没有必要返回他们的指针了。`make()` 函数的函数签名如下：

```
1 | func make(t Type, size ...IntegerType) Type
```

```
1 | var userinfo map[string]string
2 | userinfo = make(map[string]string)
3 | userinfo["username"] = "张三"
4 | fmt.Println(userinfo)
```

9.3.3 new() 与 make() 的区别

1. 二者都是用来做内存分配的。
2. `make()` 只用于 `slice`、`map` 以及 `channel` 的初始化，返回的还是这三个引用类型本身。
3. 而 `new()` 用于类型的内存分配，并且内存对应的值为类型零值，返回的是指向类型的指针。

10. 结构体

Go 语言中没有类的概念，Go 语言中的结构体和其他语言中的类有点相似。和其他面向对象语言中的类相比，Go 语言中的结构体具有更高的扩展性和灵活性。

10.1 自定义类型和类型别名

10.1.1 自定义类型

在 Go 语言中有一些基本的数据类型，如 `string`、`int`、`float`、`bool` 等数据类型，Go 语言中可以使用 `type` 关键字来定义自定义类型。

```
1 | type myInt int
```

10.1.2 类型别名

`TypeAlias` 只是 `Type` 的别名，本质上 `TypeAlias` 与 `Type` 是同一个类型。

```
1 type TypeAlias = Type
```

`rune` 和 `byte` 就是类型别名，他们的底层定义如下

```
1 type byte = uint8
2 type rune = int32
```

10.1.3 自定义类型和类型别名的区别

```
1 package main
2
3 import "fmt"
4
5 // 类型定义
6 type newInt int
7
8 // 类型别名
9 type myInt = int
10
11 func main() {
12     var a newInt
13     var b myInt
14     fmt.Printf("type of a:%T\n", a) // type of a:main.newInt
15     fmt.Printf("type of b:%T\n", b) // type of b:int
16 }
17
```

10.2 结构体的初始化

10.2.1 结构体的声明

使用 `type` 和 `struct` 关键字来定义结构体，具体代码格式如下：

```
1 type structName struct {
2     fieldName fieldType
3     ...
4 }
```

其中：

- `structName`：类型名，表示自定义结构体的名称，在同一个包内不能重复。
- `fieldName`：字段名，表示结构体字段名。结构体中的字段名必须唯一。
- `fieldType`：字段类型，表示结构体字段的具体类型。

```
1 type person struct {
2     name string
3     city string
4     age int8
5 }
```

⚠ Caution

结构体首字母可以大写也可以小写，大写表示这个结构体是公有的，在其他的包里面可以使用。小写表示这个结构体是私有的，只有这个包里面才能使用。

10.2.2 结构体实例化

只有当结构体实例化时，才会真正地分配内存。也就是必须实例化后才能使用结构体的字段。可以使用 `var` 关键字声明结构体类型。

```
1 package main
2
3 import "fmt"
4
5 type person struct {
6     name string
7     city string
8     age int
9 }
10
11 func main() {
12     var p1 person
13     p1.name = "张三"
14     p1.city = "北京"
15     p1.age = 18
16     fmt.Printf("p1=%v\n", p1) // p1={张三 北京 18}
17     fmt.Printf("p1=%#v\n", p1) // p1=main.person{name:"张三", city:"北京", age:18}
18 }
```

我们还可以通过使用 `new()` 对结构体进行实例化，得到的是结构体的地址。格式如下：

```
1 package main
2
3 import "fmt"
4
5 type person struct {
6     name string
7     city string
8     age int
9 }
10
11 func main() {
12     var p2 = new(person)
13     p2.name = "张三"
```

```

14     p2.age = 20
15     p2.city = "北京"
16     fmt.Printf("%T\n", p2) // *main.person
17     fmt.Printf("p2=%#v\n", p2) // p2=&main.person{name:"张三", city:"北京", age:20}
18 }

```

⚠ Caution

在 GoLang 中支持对结构体指针直接使用 `.` 来访问结构体的成员。`p2.name = "张三"` 其实在底层是 `(*p2).name = "张三"`

也可以通过键值对初始化的方法。格式如下：

```

1  package main
2
3  import "fmt"
4
5  type person struct {
6      name string
7      city string
8      age int
9  }
10
11 func main() {
12     var p3 = &person{
13         name: "zhangsan",
14         city: "北京",
15         age: 18,
16     }
17     fmt.Printf("p3=%#v\n", p3) //p3=&main.person{name:"zhangsan", city:"北京",
18     age:18}
19 }

```

10.3 结构体方法和接收者

在 Go 语言中，没有类的概念但是可以给类型（结构体，自定义类型）定义方法。所谓方法就是定义了接收者的函数。

接收者的概念就类似于其他语言中的 `this` 或者 `self`。

```

1 func (ReceiveVar ReceiveType) funcName (para Type) (return Type) {
2     funcBody
3 }

```

其中：

- **ReceiveVar**：接收者变量，接收者中的参数变量名在命名时，官方建议使用接收者类型名的第一个小写字母，而不是 `self`、`this` 之类的命名。例如，`Person` 类型的接收者变量应该命名为 `p`，`Connector` 类型的接收者变量应该命名为 `c` 等。
- **ReceiveType**：接收者类型，接收者类型和参数类似，可以是指针类型和非指针类型。

```

1  package main
2
3  import "fmt"
4
5  type Person struct {
6      name string
7      age  int
8      sex  string
9  }
10
11 func (p Person) PrintInfo() {
12     fmt.Println("name:", p.name)
13     fmt.Println("age:", p.age)
14     fmt.Println("sex:", p.sex)
15 }
16
17 func main() {
18     var p1 Person
19     p1.name = "张三"
20     p1.age = 18
21     p1.sex = "男"
22
23     p1.PrintInfo()
24 }
25
26 // 输出:
27 // name: 张三
28 // age: 18
29 // sex: 男

```

值类型作为接收者时，Go 语言会在代码运行时将接收者的值复制一份。在值类型接收者的方法中可以获取接收者的成员值，但修改操作只是针对副本，无法修改接收者变量本身。

指针类型作为接收者时，由于指针的特性，调用方法时修改接收者指针的任意成员变量，在方法结束后，修改都是有效的。这种方式就十分接近于其他语言中面向对象中的 `this` 或者 `self`。

```

1  package main
2
3  import "fmt"
4
5  type Person struct {
6      name string
7      age  int
8      sex  string
9  }
10
11 // PrintInfo 值类型接受者
12 func (p Person) PrintInfo() {
13     fmt.Printf("Person %v, %d, %v\n", p.name, p.age, p.sex)
14 }
15
16 // SetInfo 指针类型接受者
17 func (p *Person) SetInfo(name string, age int, sex string) {

```

```

18     p.name = name
19     p.age = age
20     p.sex = sex
21 }
22
23 func main() {
24     var p1 Person
25     p1.name = "张三"
26     p1.age = 18
27     p1.sex = "男"
28
29     p1.PrintInfo() // Person 张三, 18, 男
30     p1.SetInfo("李四", 19, "女")
31     p1.PrintInfo() // Person 李四, 19, 女
32 }

```

⚠ Caution

Go 文档不推荐使用结构体在值接收器和指针接收器上都有方法。

10.4 给任意类型添加方法

在 Go 语言中，接收者的类型可以是任何类型，不仅仅是结构体，任何类型都可以拥有方法。

```

1  package main
2
3  import "fmt"
4
5  type myInt int
6
7  func (i myInt) SayHello() {
8      fmt.Printf("Hello, World!\n")
9  }
10
11 func main() {
12     var m1 myInt
13     m1.SayHello() // Hello, World!
14     m1 = 100
15     fmt.Printf("%#v, %T\n", m1, m1) // 100, main.myInt
16 }

```

⚠ Caution

非本地类型不能定义方法，也就是说我们不能给别的包的类型定义方法。

10.5 结构体的匿名字段

结构体允许其成员字段在声明时没有字段名而只有类型，这种没有名字的字段就称为匿名字段。

```
1 //Person 结构体 Person 类型
2 type Person struct {
3     string
4     int
5 }
6
7 func main() {
8     p1 := Person{
9         "小王子",
10        18,
11    }
12    fmt.Printf("%#v\n", p1) // main.Person{string:"北京", int:18}
13    fmt.Println(p1.string, p1.int) // 北京 18
14 }
```

匿名字段默认采用类型名作为字段名，结构体要求字段名称必须唯一，因此一个结构体中同种类型的匿名字段只能有一个。

10.6 嵌套结构体

一个结构体中可以嵌套包含另一个结构体或结构体指针。

```
1 type Address struct {
2     Province string
3     City      string
4 }
5
6 type User struct {
7     Name      string
8     Gender    string
9     Address   Address
10 }
11
12 func main() {
13     user1 := User{
14         Name:    "张三",
15         Gender:  "男",
16         Address: Address{
17             Province: "广东",
18             City:     "深圳",
19         },
20     }
21     fmt.Printf("%#v\n", user1)
22 }
23
24 // 输出:
25 // main.User{Name:"张三", Gender:"男",
```



```
26 // Address:main.Address{Province:"广东", City:"深圳"}}
```

10.7 结构体的继承

Go 语言中使用结构体也可以实现其他编程语言中的继承。

```
1  type Animal struct {
2      name string
3  }
4
5  func (a Animal) Eat() {
6      fmt.Printf("Animal %s eats!\n", a.name)
7  }
8
9  type Dog struct {
10     Age int
11     Animal
12 }
13
14 func main() {
15     dog1 := Dog{
16         Age: 1,
17         Animal: Animal{
18             name: "旺财",
19         },
20     }
21     dog1.Eat()
22 }
23
24 // 输出:
25 // Animal 旺财 eats!
```

11. JSON

11.1 JSON 序列化和反序列化

GoLang 中的序列化和反序列化主要通过 `"encoding/json"` 包中的 `json.Marshal()` 和 `json.Unmarshal()` 方法实现。

11.1.1 序列化

```
1  package main
2
3  import (
4      "encoding/json"
5      "fmt"
6  )
```

```

7
8 type Student struct {
9     ID      int
10    name    string // 私有属性不能被 json 包访问
11    Age     int
12    Gender  string
13 }
14
15 func main() {
16     var s Student = Student{
17         ID:      1,
18         name:    "张三", // 私有属性不能被 json 包访问
19         Age:     18,
20         Gender:  "男",
21     }
22
23     sByte, err := json.Marshal(s)
24     if err != nil {
25         // 处理错误
26         fmt.Println(err)
27     }
28     sStr := string(sByte)
29     fmt.Println(sStr) // {"ID":1,"Age":18,"Gender":"男"}
30 }

```

11.1.2 反序列化

```

1 package main
2
3 import (
4     "encoding/json"
5     "fmt"
6 )
7
8 type Student struct {
9     ID      int
10    Name    string
11    Age     int
12    Gender  string
13 }
14
15 func main() {
16     var jsonStr = `{"ID":1,"Name":"李四","Age":18,"Gender":"男"}`
17     var student Student
18     err := json.Unmarshal([]byte(jsonStr), &student)
19     if err != nil {
20         fmt.Println(err)
21     }
22     fmt.Printf("student:%#v\n", student) // student:main.Student{ID:1, Name:"李四",
Age:18, Gender:"男"}
23 }

```

11.2 结构体标签 Tag

Tag 是结构体的元信息，可以在运行的时候通过反射的机制读取出来。Tag 在结构体字段的后方定义，由一对反引号包裹起来，具体的格式如下：

```
1 | `key1 : "value1"`
```

⚠ Caution

为结构体编写 Tag 时，必须严格遵守键值对的规则。结构体标签的解析代码的容错能力很差，一旦格式写错，编译和运行时都不会提示任何错误，通过反射也无法正确取值。例如不要在 key 和 value 之间添加空格。

```
1 | type Student struct {
2 |     ID    int    `json:"id"`
3 |     Name  string `json:"name"`
4 |     Age   int    `json:"age"`
5 |     Gender string `json:"gender"`
6 | }
```

11.3 嵌套结构体的 JSON

```
1 | package main
2 |
3 | import (
4 |     "encoding/json"
5 |     "fmt"
6 | )
7 |
8 | type Student struct {
9 |     ID    int    `json:"id"`
10 |    Name  string `json:"name"`
11 |    Age   int    `json:"age"`
12 |    Gender string `json:"gender"`
13 | }
14 |
15 | type Class struct {
16 |     Name      string    `json:"name"`
17 |     Students []Student `json:"students"`
18 | }
19 |
20 | func main() {
21 |     c := &Class{
22 |         Name:      "1班",
23 |         Students: make([]Student, 0, 200),
24 |     }
25 |
26 |     for i := 0; i < 10; i++ {
27 |         stu := Student{
```

```
28         ID:    i + 1,
29         Name:   fmt.Sprintf("学号%d", i+1),
30         Age:    18,
31         Gender: "男",
32     }
33     c.Students = append(c.Students, stu)
34 }
35
36 // 序列化
37 data, err := json.Marshal(c)
38 if err != nil {
39     fmt.Println(err)
40 }
41 fmt.Printf("class:%#v\n", c)
42 fmt.Printf("data:%#v\n", string(data))
43 }
```

12. 包

包 (package) 是多个 Go 源码的集合，是一种高级的代码复用方案。Golang 中的包可以分为三种：1、系统内置包；2、自定义包；3、第三方包。

12.1 包管理工具 go mod

Go1.11 版本之后无需手动配置环境变量，使用 `go mod` 管理项目，也不需要非得把项目放到 `$GOPATH` 指定目录下，你可以在你磁盘的任何位置新建一个项目，Go1.13 以后可以彻底不要 `$GOPATH` 了。

12.1.1 go mod init 初始化项目

实际项目开发中我们首先要在我们项目目录中用 `go mod` 命令生成一个 `go.mod` 文件管理我们项目的依赖。

```
1 | go mod init demo21
```

12.1.2 其他命令

命令	用途	常用参数
<code>go mod init</code>	初始化模块	
<code>go mod tidy</code>	整理依赖	
<code>go mod download</code>	下载依赖	
<code>go mod graph</code>	查看依赖图	
<code>go mod why</code>	查看依赖原因	<code>-m</code> 模块级别
<code>go mod vendor</code>	创建 vendor 目录	

12.2 自定义包

包 (package) 是多个 Go 源码的集合，一个包可以简单理解为一个存放多个 `.go` 文件的文件夹。该文件夹下面的所有 `go` 文件都要在代码的第一行添加如下代码，声明该文件归属的包。

```
1 package Name
```

⚠ Caution

- 一个文件夹下面直接包含的文件只能归属一个 `package`， 同样一个 `package` 的文件不能在多个文件夹下。
- 包名可以不和文件夹的名字一样， 包名不能包含 `-` 符号。
- 包名为 `main` 的包为应用程序的入口包，这种包编译后会得到一个可执行文件，而编译不包含 `main` 包的源代码则不会得到可执行文件。

12.2.1 定义包

在当前目录下 `mkdir` 一个 `test` 文件夹，同时在该文件夹下 `touch` 一个 `calc.go` 文件。这样表示一个名为 `test` 的包被创建了。

```
1 mkdir test
2 cd test && touch calc.go
```

```

1  package test
2
3  var Age = 18 // 首字母大写表示公有属性
4  var name = "小王" // 首字母小写表示私有属性
5
6  func Add(x, y int) int {
7      return x + y
8  }
9
10 func Sub(x, y int) int {
11     return x - y
12 }

```

12.2.2 导入包

单行导入的格式如下：

```

1  import "package 1"
2  import "package 2"

```

多行导入的格式如下：

```

1  import (
2      "package 1"
3      "package 2"
4  )

```

如果只希望导入包，而不使用包内部的数据时，可以使用匿名导入包。具体的格式如下：

```

1  import _ "package path"

```

在导入包名的时候，我们还可以为导入的包设置别名。通常用于导入的包名太长或者导入的包名冲突的情况。具体语法格式如下：

```

1  import alias "package path"

```

具体案例：

```

1  package main
2
3  import (
4      "demo21/test"
5      "fmt"
6  )
7
8  func main() {
9      add := test.Add(10, 20)
10     fmt.Println(add)
11
12     sub := test.Sub(10, 20)

```

```
13     fmt.Println(sub)
14 }
```

12.3 init() 初始化函数

在 Go 语言程序执行时导入包语句会自动触发包内部 `init()` 函数的调用。需要注意的是：`init()` 函数没有参数也没有返回值。`init()` 函数在程序运行时自动被调用执行，不能在代码中主动调用它。

```
1  package main
2
3  import "fmt"
4
5  var x int = 10
6
7  const pi = 3.14
8
9  func init() {
10     fmt.Printf("x is %d\n", x)
11 }
12
13 func main() {
14     fmt.Printf("pi is %.2f\n", pi)
15 }
16
17 // 输出:
18 // x is 10
19 // pi is 3.14
```

12.4 第三方包

我们可以在 <https://pkg.go.dev/> 查找看常见的 Golang 第三方包。

12.4.1 初始化项目

```
1 | go mod init project
```

12.4.2 下载安装这个包（非必须）

例如要下载解决 `float` 精度损失的包 `decimal`。

```
1 | go get github.com/shopspring/decimal
```

12.4.3 go mod tidy 下载丢失的包

`go mod tidy` 命令可以增加丢失的 `module`，同时去掉未用的 `module`（推荐）

```
1 | go mod tidy
```

13. 接口

13.1 接口的声明

在 Golang 中接口（interface）是一种类型，一种抽象的类型。接口（interface）是一组函数 method 的集合，Golang 中的接口不能包含任何变量。

在 Golang 中接口中的所有方法都没有方法体，接口定义了一个对象的行为规范，只定义规范不实现。接口体现了程序设计的多态和高内聚低耦合的思想。

接口的定义格式如下：

```
1 | type interfacName interface{
2 |     func1 (para) return 1
3 |     func2 (para) return 2
4 |     ...
5 | }
```

其中：

- `interfacName`：接口名，使用 `type` 将接口定义为自定义的类型名。Go 语言的接口在命名时，一般会在单词后面添加 `er`，如有写操作的接口叫 `Writer`，有字符串功能的接口叫 `Stringer` 等。接口名最好要能突出该接口的类型含义。
- `func`：当方法名首字母是大写且这个接口类型名首字母也是大写时，这个方法可以被接口所在的包（package）之外的代码访问。
- `para、return`：参数列表、返回值列表，参数列表和返回值列表中的参数变量名可以省略。

```
1 | package main
2 |
3 | import "fmt"
4 |
5 | type Usber interface {
6 |     Start()
7 |     Stop()
8 | }
9 |
10 | type Phone struct {
11 |     Name string
12 | }
13 |
```



```

14 func (p Phone) Start() {
15     fmt.Println(p.Name, "手机开始工作")
16 }
17
18 func (p Phone) Stop() {
19     fmt.Println(p.Name, "手机停止工作")
20 }
21
22 type Camera struct {
23     Name string
24 }
25
26 func (c Camera) Start() {
27     fmt.Println(c.Name, "相机开始工作")
28 }
29
30 func (c Camera) Stop() {
31     fmt.Println(c.Name, "相机停止工作")
32 }
33
34 func main() {
35     // 创建手机对象
36     phone := Phone{"小米"}
37     var usbPhone Usber = phone // 表示 phone 实现了 Usber 接口
38     usbPhone.Start() // 小米 手机开始工作
39     usbPhone.Stop() // 小米 手机停止工作
40
41     camera := Camera{"尼康"}
42     var usbCamera Usber = camera // 表示 camera 实现了 Usber 接口
43     usbCamera.Start() // 尼康 相机开始工作
44     usbCamera.Stop() // 尼康 相机停止工作
45 }

```

⚠ Caution

Golang 中的接口也是一种数据类型，不需要显示实现。只需要一个变量含有接口类型中的所有方法，那么这个变量就实现了这个接口。

13.2 空接口

Golang 中的接口可以不定义任何方法，没有定义任何方法的接口就是空接口。空接口表示没有任何约束，因此任何类型变量都可以实现空接口。用空接口可以表示任意数据类型。

```

1 func main() {
2     // 定义一个空接口 x, x 变量可以接收任意的数据类型
3     var x interface{}
4     s := "你好 golang"
5     x = s
6     fmt.Printf("type:%T value:%v\n", x, x)
7     i := 100
8     x = i
9     fmt.Printf("type:%T value:%v\n", x, x)
10    b := true
11    x = b
12    fmt.Printf("type:%T value:%v\n", x, x)
13 }

```

Tip

`any()` 是空接口的别名，在所有方面都等效于空接口。

13.2.1 空接口作为函数的参数

使用空接口实现可以接收任意类型的函数参数。

```

1 func show(a interface{}) {
2     fmt.Printf("type:%T value:%v\n", a, a)
3 }

```

13.2.2 map 的值实现空接口

使用空接口实现可以保存任意值的字典。

```

1 var studentInfo = make(map[string]interface{})
2 studentInfo["name"] = "张三"
3 studentInfo["age"] = 18
4 studentInfo["married"] = false
5 studentInfo["hobby"] = []string{"football", "swimming"}
6 // 错误写法:
7 // invalid operation: cannot index studentInfo["hobby"] (map index expression of type
  interface{})
8 // fmt.Println(studentInfo["hobby"][1])
9 // 正确写法:
10 fmt.Println(studentInfo["hobby"].([]string)[1])

```

13.2.3 切片实现空接口

```

1 var slice = []interface{}{"张三", 20, true, 32.2}
2 fmt.Println(slice)

```

13.3 类型断言

一个接口的值（简称接口值）是由一个具体类型和具体类型的值两部分组成的。这两部分分别称为接口的动态类型和动态值。

```
1 | x.(T)
```

其中：

- `x`：表示类型为 `interface{}` 的变量
- `T`：表示断言 `x` 可能是的类型

该语法返回两个参数，第一个参数是 `x` 转化为 `T` 类型后的变量，第二个值是一个布尔值，若为 `true` 则表示断言成功，为 `false` 则表示断言失败。

```
1 | var x interface{}
2 | x = "Hello world"
3 | v, ok := x.(string)
4 | if ok {
5 |     fmt.Println(v)
6 | } else {
7 |     fmt.Println("类型断言失败")
8 | }
```

⚠ Caution

`x.(type)` 语句也可以判断一个变量的类型，但是只能结合 `switch` 语句使用

```
1 | func justifyType(x interface{}) {
2 |     switch v := x.(type) {
3 |     case string:
4 |         fmt.Printf("x is a string, value is %v\n", v)
5 |     case int:
6 |         fmt.Printf("x is a int is %v\n", v)
7 |     case bool:
8 |         fmt.Printf("x is a bool is %v\n", v)
9 |     default:
10 |        fmt.Println("unsupport type", v)
11 |     }
12 | }
```

📌 Note

只有当有两个或两个以上的具体类型必须以相同的方式进行处理时才需要定义接口。不要为了接口而写接口，那样只会增加不必要的抽象，导致不必要的运行时损耗。

13.4 结构体值接收者和指针接收者实现接口的区别

13.4.1 值接收者

如果结构体中的方法是值接收者，那么实例化后的结构体值类型和结构体指针类型都可以赋值给接口变量。

```
1  type Usber interface {
2      Start()
3      Stop()
4  }
5
6  type Phone struct {
7      Name string
8  }
9
10 func (p Phone) Start() {
11     fmt.Println(p.Name, "手机开始工作")
12 }
13
14 func (p Phone) Stop() {
15     fmt.Println(p.Name, "手机停止工作")
16 }
17
18 func main() {
19     // 正确写法
20     phone1 := Phone{"小米"}
21     var p1 Usber = phone1
22     p1.Start() // 小米 手机开始工作
23     p1.Stop()  // 小米 手机停止工作
24
25     // 正确写法
26     phone2 := &Phone{"苹果"}
27     var p2 Usber = phone2
28     p2.Start() // 苹果 手机开始工作
29     p2.Stop()  // 苹果 手机停止工作
30 }
```

13.4.2 指针接收者

如果结构体中的方法是指针接收者，那么实例化后结构体指针类型都可以赋值给接口变量，**结构体值类型没法赋值给接口变量。**

```
1  type Usber interface {
2      Start()
3      Stop()
4  }
5
6  type Phone struct {
7      Name string
8  }
9
```

```

10 func (p *Phone) Start() {
11     fmt.Println(p.Name, "手机开始工作")
12 }
13
14 func (p *Phone) Stop() {
15     fmt.Println(p.Name, "手机停止工作")
16 }
17
18 func main() {
19     // 错误写法
20     phone1 := Phone{"小米"}
21     var p1 Usber = phone1
22     p1.Start() // 小米 手机开始工作
23     p1.Stop()  // 小米 手机停止工作
24
25     // 正确写法
26     phone2 := &Phone{"苹果"}
27     var p2 Usber = phone2
28     p2.Start() // 苹果 手机开始工作
29     p2.Stop()  // 苹果 手机停止工作
30 }

```

13.5 一个结构体实现多个接口

```

1  package main
2
3  import "fmt"
4
5  type A interface {
6      Get() string
7  }
8
9  type B interface {
10     Set(string, int)
11 }
12
13 type People struct {
14     Name string
15     Age  int
16 }
17
18 func (p *People) Get() string {
19     return fmt.Sprintf("name:%s, age:%d", p.Name, p.Age)
20 }
21
22 func (p *People) Set(name string, age int) {
23     p.Name = name
24     p.Age = age
25 }
26
27 func main() {
28     var people = &People{"张三", 18}

```

```

29
30     var p1 A = people // 实现 A 接口
31     var p2 B = people // 实现 B 接口
32
33     fmt.Println(p1.Get()) // name:张三, age:18
34     p2.Set("王五", 20)
35     fmt.Println(p1.Get()) // name:王五, age:20
36 }

```

13.6 接口嵌套

```

1  package main
2
3  import "fmt"
4
5  type Sayer interface {
6      Say()
7  }
8
9  type Mover interface {
10     Move()
11 }
12
13 type Animal interface {
14     Sayer
15     Mover
16 }
17
18 type Cat struct {
19     Name string
20 }
21
22 func (c Cat) Say() {
23     fmt.Println(c.Name, "喵喵喵")
24 }
25
26 func (c Cat) Move() {
27     fmt.Println(c.Name, "跑")
28 }
29
30 func main() {
31     var cat = Cat{"小猫"}
32     var animal Animal = cat
33     animal.Say() // 小猫 喵喵喵
34     animal.Move() // 小猫 跑
35 }

```

14. 协程

14.1 高并发基础

14.1.1 线程与进程

- **进程 (Process)** 就是程序在操作系统中的一次执行过程，是系统进行资源分配和调度的基本单位，进程是一个动态概念，是程序在执行过程中分配和管理资源的基本单位，每一个进程都有一个自己的地址空间。一个进程至少有 5 种基本状态，它们是：初始态、执行态、等待状态、就绪状态以及终止状态。
- **线程** 是进程的一个执行实例，是程序执行的最小单元，它是比进程更小的能独立运行的基本单位。一个进程可以创建多个线程，同一个进程中的多个线程可以并发执行，一个程序要运行的话至少有一个进程。

14.1.2 并行与并发

- **并发**：多个线程同时竞争一个位置，竞争到的才可以执行，每一个时间段只有一个线程在执行。
- **并行**：多个线程可以同时执行，每一个时间段，可以有多个线程同时执行。

通俗的讲多线程程序在单核 CPU 上面运行就是并发，多线程程序在多核 CPU 上运行就是并行，如果线程数大于 CPU 核数，则多线程程序在多个 CPU 上面运行既有并行又有并发。

14.2 Goroutine 的定义

Golang 中的主线程：（可以理解为线程/也可以理解为进程），在一个 Golang 程序的主线程上可以起多个协程。Golang 中多协程可以实现并行或者并发。

协程：可以理解为用户级线程，这是对内核透明的，也就是系统并不知道有协程的存在，是完全由用户自己的程序进行调度的。

Golang 的一大特色就是从语言层面原生支持协程，在函数或者方法前面加 `go` 关键字就可创建一个协程。可以说 Golang 中的协程就是 goroutine 。

Tip

Golang 中的多协程有点类似其他语言中的多线程。

多协程和多线程：Golang 中每个 goroutine（协程）默认占用内存远比 Java、C 的线程少。OS 线程（操作系统线程）一般都有固定的栈内存（通常为 2MB 左右），一个 goroutine（协程）占用内存非常小，只有 2KB 左右，多协程 goroutine 切换调度开销方面远比线程要少。

14.3 Goroutine 的使用

```
1 package main
2
3 import (
4     "fmt"
```

```

5     "strconv"
6     "time"
7 )
8
9 func test() {
10     for i := 1; i ≤ 10; i++ {
11         fmt.Println("test () hello, world " + strconv.Itoa(i))
12         time.Sleep(time.Second)
13     }
14 }
15
16 func main() {
17     go test() // 开启了一个协程
18     for i := 1; i ≤ 10; i++ {
19         fmt.Println(" main() hello, golang" + strconv.Itoa(i))
20         time.Sleep(time.Second)
21     }
22 }

```

上面代码看上去没有问题，但是要注意主线程执行完毕后即使协程没有执行完毕，程序会退出，所以我们需要对上面代码进行改造。 `sync.WaitGroup` 可以实现主线程等待协程执行完毕。

```

1  package main
2
3  import (
4      "fmt"
5      "strconv"
6      "sync"
7      "time"
8  )
9
10 var wg sync.WaitGroup // 1. 定义全局的 WaitGroup
11
12 func test() {
13     for i := 1; i ≤ 10; i++ {
14         fmt.Println("test () 你好 golang " + strconv.Itoa(i))
15         time.Sleep(time.Millisecond * 50)
16     }
17     wg.Done() // 4. goroutine 结束就登记-1
18 }
19
20 func main() {
21     wg.Add(1) // 2. 启动一个 goroutine 就登记 +1
22     defer wg.Wait() // 3. 等待所有 goroutine 结束
23     go test() // 开启了一个协程
24     for i := 1; i ≤ 2; i++ {
25         fmt.Println(" main() 你好 golang" + strconv.Itoa(i))
26         time.Sleep(time.Millisecond * 50)
27     }
28 }

```

多次执行上面的代码，会发现每次打印的数字的顺序都不一致。这是因为 10 个 goroutine 是并发执行的，而 goroutine 的调度是随机的。

14.4 设置 CPU 数量

Go 运行时的调度器使用 `GOMAXPROCS` 参数来确定需要使用多少个 OS 线程来同时执行 Go 代码。默认值是机器上的 CPU 核心数。例如在一个 8 核心的机器上，调度器会把 Go 代码同时调度到 8 个 OS 线程上。

Go 语言中可以通过 `runtime.GOMAXPROCS()` 函数设置当前程序并发时占用的 CPU 逻辑核心数。

```
1 package main
2
3 import (
4     "fmt"
5     "runtime"
6 )
7
8 func main() {
9     // 获取当前计算机上面的 CPU 个数
10    cpuNum := runtime.NumCPU()
11    fmt.Println("cpuNum=", cpuNum) // cpuNum= 20
12
13    // 可以自己设置使用多个 cpu
14    runtime.GOMAXPROCS(cpuNum - 1)
15    fmt.Println("ok")
16 }
```

14.5 管道

管道 (Channel) 是 Golang 在语言级别上提供的 goroutine 间的通讯方式，我们可以使用 `channel` 在多个 goroutine 之间传递消息。如果说 goroutine 是 Go 程序并发的执行体，`channel` 就是它们之间的连接。`channel` 是可以让一个 goroutine 发送特定值到另一个 goroutine 的通信机制。

Tip

Golang 的并发模型是 CSP (Communicating Sequential Processes)，提倡通过通信共享内存而不是通过共享内存而实现通信。

Go 语言中的 `channel` 是一种特殊的类型，总是遵循先入先出 (First In First Out) 的规则，保证收发数据的顺序。

14.5.1 管道的声明

`channel` 是一种类型，一种引用类型。声明管道类型的格式如下：

```
1 var ch1 chan Type
```

声明的管道后需要使用 `make()` 函数初始化之后才能使用。

```
1 make(chan Type, cap)
```

14.5.2 管道的操作

管道有发送（send）、接收（receive）和关闭（close）三种操作。发送和接收都使用  符号。

```
1 package main
2
3 import "fmt"
4
5 func main() {
6     // 创建一个带缓冲的 channel，容量为 1
7     ch := make(chan int, 10)
8
9     // 1. 发送：把 10 发送到 ch 中
10    ch <- 10
11
12    // 2. 接收：从 ch 中接收数据
13    i := <-ch
14    fmt.Println(i)
15
16    // 3. 关闭：关闭 channel
17    close(ch)
18 }
```

关于关闭管道需要注意的事情是，只有在通知接收方 goroutine 所有的数据都发送完毕的时候才需要关闭管道。管道是可以被垃圾回收机制回收的，它和关闭文件是不一样的，在结束操作之后关闭文件是必须要做的，但关闭管道不是必须的。

⚠ Caution

1. 对一个关闭的管道再发送值就会导致 `panic`。
2. 对一个关闭的管道进行接收会一直获取值直到管道为空。
3. 对一个关闭的并且没有值的管道执行接收操作会得到对应类型的零值。
4. 关闭一个已经关闭的管道会导致 `panic`。

14.5.3 管道阻塞

如果创建管道的时候没有指定容量，那么我们可以叫这个管道为**无缓冲的管道**。无缓冲的管道又称为阻塞的管道。我们来看一下下面的代码：

```
1 func main() {
2     ch := make(chan int)
3     ch <- 10
4     fmt.Println("发送成功")
5 }
```

面这段代码能够通过编译，但是执行的时候会出现以下错误：

```
1 fatal error: all goroutines are asleep - deadlock!
```

解决上面问题的方法还有一种就是使用有缓冲区的管道。我们可以在使用 `make()` 函数初始化管道的时候为其指定管道的容量，例如：

```
1 func main() {
2     ch := make(chan int, 1) // 创建一个容量为 1 的有缓冲区管道
3     ch <- 10
4     fmt.Println("发送成功")
5 }
```

只要管道的容量大于零，那么该管道就是有缓冲的管道，管道的容量表示管道中能存放元素的数量。

管道阻塞具体代码如下：

```
1 func main() {
2     ch := make(chan int, 1)
3     ch <- 10
4     ch <- 12
5     fmt.Println("发送成功")
6 }
7
8 // 输出：
9 // fatal error: all goroutines are asleep - deadlock!
```

解决办法：

```
1 func main() {
2     ch := make(chan int, 1)
3     ch <- 10 // 放进去
4     <-ch // 取走
5     ch <- 12 // 放进去
6     <-ch // 取走
7     ch <- 17 // 还可以放进去
8     fmt.Println("发送成功")
9 }
10
11 // 输出：
12 // 发送成功
```

14.5.4 遍历管道

当向管道中发送完数据时，我们可以通过 `close()` 函数来关闭管道。当管道被关闭时，再往该管道发送值会引发 `panic`，从该管道取值的操作会先取完管道中的值，再然后取到的值一直都是对应类型的零值。可以通过 `for` 或 `for range` 语句遍历管道。

```

1  var ch1 = make(chan int, 5)
2
3  for i := 0; i < 5; i++ {
4      ch1 ← i + 1
5  }
6
7  close(ch1) // 关闭 channel
8
9  for val := range ch1 {
10     fmt.Println(val)
11 }

```

⚠ Caution

使用 `for range` 遍历管道，当管道被关闭的时候就会退出 `for range`，如果没有关闭管道就会报错误。

```

1  var ch2 = make(chan int, 5)
2
3  for i := 0; i < 5; i++ {
4      ch2 ← i + 1
5  }
6
7  for i := 0; i < 5; i++ {
8      fmt.Println(←ch2)
9  }

```

📌 Note

使用 `for` 遍历管道，不关闭管道，也不会报错。

14.6 单向管道

有的时候我们会将管道作为参数在多个任务函数间传递，很多时候我们在不同的任务函数中使用管道都会对其进行限制，比如限制管道在函数中只能发送或只能接收。

```

1  package main
2
3  import "fmt"
4
5  func main() {
6      // 1. 在默认情况下，管道是双向
7      var ch1 chan int // 可读可写
8      ch1 = make(chan int, 3)
9      fmt.Printf("%v\n", ch1)
10
11     // 2. 声明为只写
12     var ch2 chan← int
13     ch2 = make(chan int, 3)

```

```

14     ch2 ← 20
15     //fmt.Println(←ch2)
16     //invalid operation: cannot receive from send-only channel ch2 (variable of type
chan← int)
17
18     // 3. 声明为只读
19     var ch3 ←chan int
20     ch3 = make(chan int, 3)
21     // ch3 ← 20
22     // invalid operation: cannot send to receive-only channel ch3 (variable of type
←chan int)
23     num := ←ch3
24     fmt.Println(num)
25 }

```

14.7 select

在某些场景下我们需要同时从多个通道接收数据。这个时候就可以用到 Golang 中给我们提供的 `select` 多路复用。通常情况通道在接收数据时，如果没有数据可以接收将会发生阻塞。

比如说下面代码来实现从多个通道接受数据的时候就会发生阻塞：

```

1  for {
2      // 尝试从 ch1 接收值
3      data, ok := ←ch1
4      // 尝试从 ch2 接收值
5      data, ok := ←ch2
6      ...
7  }

```

`select` 的使用类似于 `switch` 语句，它有一系列 `case` 分支和一个默认的分支。每个 `case` 会对应一个管道的通信（接收或发送）过程。`select` 会一直等待，直到某个 `case` 的通信操作完成时，就会执行 `case` 分支对应的语句。具体格式如下：

```

1  select{
2      case ←ch1:
3          ...
4      case data := ←ch2:
5          ...
6      case ch3← data:
7          ...
8      default:
9          ...
10 }

```

```

1  package main
2
3  import (
4      "fmt"
5      "time"
6  )

```

```

7
8 func main() {
9     intChan := make(chan int, 10)
10    for i := 0; i < 10; i++ {
11        intChan ← i
12    }
13
14    stringChan := make(chan string, 5)
15    for i := 0; i < 5; i++ {
16        stringChan ← "string_" + fmt.Sprintf("%d", i)
17    }
18
19    for {
20        select {
21            case v := ←intChan:
22                fmt.Println("int:", v)
23            case v := ←stringChan:
24                fmt.Println("string:", v)
25            default:
26                fmt.Printf("没有数据了")
27                time.Sleep(time.Second)
28                return
29        }
30    }
31 }

```

⚠ Caution

使用 `select` 获取管道中的数据不需要 `close()`。

14.8 并发安全

14.8.1 互斥锁

互斥锁是传统并发编程中对共享资源进行访问控制的主要手段，它由标准库 `sync` 中的 `Mutex` 结构体类型表示。`sync.Mutex` 类型只有两个公开的指针方法，`Lock()` 和 `Unlock()`。`Lock()` 锁定当前的共享资源，`Unlock()` 进行解锁。

```

1 package main
2
3 import (
4     "fmt"
5     "runtime"
6     "sync"
7     "time"
8 )
9
10 var count = 0
11 var wg sync.WaitGroup
12 var mutex sync.Mutex
13

```

```

14 func add() {
15     mutex.Lock() // 上锁
16     count++
17     fmt.Println("Count: ", count)
18     mutex.Unlock() // 开锁
19     time.Sleep(time.Millisecond)
20     wg.Done()
21 }
22
23 func main() {
24     // 设置多个CPU核心来增加并发竞争的可能性
25     runtime.GOMAXPROCS(runtime.NumCPU())
26
27     for i := 0; i < 100; i++ {
28         wg.Add(1)
29         go add()
30     }
31     wg.Wait()
32     fmt.Printf("Final count value: %d\n", count)
33 }

```

使用互斥锁能够保证同一时间有且只有一个 `goroutine` 进入临界区，其他的 `goroutine` 则在等待锁；当互斥锁释放后，等待的 `goroutine` 才可以获取锁进入临界区，多个 `goroutine` 同时等待一个锁时，唤醒的策略是随机的。

14.8.2 读写互斥锁

互斥锁的本质是当一个 `goroutine` 访问的时候，其他 `goroutine` 都不能访问。这样在资源同步，避免竞争的同时也降低了程序的并发性能。程序由原来的并行执行变成了串行执行。

Tip

其实，当我们对一个不会变化的数据只做读操作的话，是不存在资源竞争的问题的。因为数据是不变的，不管怎么读取，多少 `goroutine` 同时读取，都是可以的。

读写锁可以让多个读操作并发，同时读取，但是对于写操作是完全互斥的。也就是说，当一个 `goroutine` 进行写操作的时候，其他 `goroutine` 既不能进行读操作，也不能进行写操作。

Golang 中的读写锁由结构体类型 `sync.RWMutex` 表示。此类型的方法集合中包含两对方法：

```

1 // 1. 写锁定和写解锁
2 func (*RWMutex) Lock()
3 func (*RWMutex) Unlock()
4
5 // 2. 读锁定和读解锁
6 func (*RWMutex) RLock()
7 func (*RWMutex) RUnlock()

```

```

1 package main
2
3 import (

```

```

4     "fmt"
5     "sync"
6     "time"
7 )
8
9 var count int
10 var mutex sync.RWMutex
11 var wg sync.WaitGroup
12
13 func write() {
14     mutex.Lock()
15     fmt.Println("执行写操作")
16     time.Sleep(time.Second * 3)
17     mutex.Unlock()
18     wg.Done()
19 }
20
21 func read() {
22     mutex.RLock()
23     fmt.Println("执行读操作")
24     time.Sleep(time.Second * 3)
25     mutex.RUnlock()
26     wg.Done()
27 }
28
29 func main() {
30     defer wg.Wait()
31
32     // 开启 10 个协程执行写操作
33     for i := 0; i < 10; i++ {
34         wg.Add(1)
35         go write()
36     }
37
38     // 开启 10 个协程执行读操作
39     for i := 0; i < 10; i++ {
40         wg.Add(1)
41         go read()
42     }
43 }

```

15. 反射

15.1 什么是发射

反射是指在程序运行期间对程序本身进行访问和修改的能力。 正常情况程序在编译时，变量被转换为内存地址，变量名不会被编译器写入到可执行部分。在运行程序时，程序无法获取自身的信息。

支持反射的语言可以在程序编译期将变量的反射信息，如字段名称、类型信息、结构体信息等整合到可执行文件中，并给程序提供接口访问反射信息，这样就可以在程序运行期获取类型的反射信息，并且有能力修改它们。

Important

Golang 中反射可以实现以下功能：

1. 反射可以在程序运行期间动态的获取变量的各种信息，比如变量的类型类别。
2. 如果是结构体，通过反射还可以获取结构体本身的信息，比如结构体的字段、结构体的方法、结构体的 tag。
3. 通过反射，可以修改变量的值，可以调用关联的方法。

Go 语言中的变量是分为两部分的：

1. **类型信息**：预先定义好的元信息。
2. **值信息**：程序运行过程中可动态变化的。

在 Golang 的反射机制中，任何接口值都由是一个具体类型和具体类型的值两部分组成的。

在 Golang 中，反射的相关功能由内置的 `reflect` 包提供，任意接口值在反射中都可以理解为由 `reflect.Type` 和 `reflect.Value` 两部分组成，并且 `reflect` 包提供了 `reflect.TypeOf()` 和 `reflect.ValueOf()` 两个重要函数来获取任意对象的 Value 和 Type。

15.2 reflect.TypeOf()

在 Go 语言中，使用 `reflect.TypeOf()` 函数可以接受任意 `interface{}` 参数，可以获得任意值的类型对象 `reflect.Type`，程序通过类型对象可以访问任意值的类型信息。

```
1 package main
2
3 import (
4     "fmt"
5     "reflect"
6 )
7
8 func ReflectFunc(x interface{}) {
9     v := reflect.TypeOf(x)
10    fmt.Println(v)
11 }
12
13 func main() {
14     // 通过 reflect 包获取变量的详细信息
15     var a int = 10
16     ReflectFunc(a) // int
17     var b string = "hello world"
18     ReflectFunc(b) // string
19 }
```

15.2.1 Name 和 Kind

在反射中的类型还分为两种：类型（Type）和种类（Kind）。因为在 Go 语言中我们可以使用 `type` 关键字构造很多自定义类型，而 `Kind` 就是指底层的类型，但在反射中，当需要区分指针、结构体等大品种的类型时，就会用到 `Kind`。举个例子，我们定义了两个指针类型和两个结构体类型，通过反射查看它们的类型和种类。

Go 语言的反射中像数组、切片、Map、指针等类型的变量，它们的 `.Name()` 都是返回空。

```
1 package main
2
3 import (
4     "fmt"
5     "reflect"
6 )
7
8 func ReflectFunc(x interface{}) {
9     v := reflect.TypeOf(x)
10    fmt.Printf("TypeOf: %v, Name: %v, Kind: %v\n", v, v.Name(), v.Kind())
11 }
12
13 type Person struct {
14     Name string
15 }
16
17 type MyInt int
18
19 func main() {
20     var a *float32 // 指针
21     var b MyInt    // 自定义类型
22     var c rune     // 类型别名
23     ReflectFunc(a) // TypeOf: *float32, Name: , Kind: ptr
24     ReflectFunc(b) // TypeOf: main.MyInt, Name: MyInt, Kind: int
25     ReflectFunc(c) // TypeOf: int32, Name: int32, Kind: int32
26
27     var d Person = Person{"Tom"}
28     ReflectFunc(d) // TypeOf: main.Person, Name: Person, Kind: struct
29
30     var e = []int{1, 2, 3}
31     ReflectFunc(e) // TypeOf: []int, Name: , Kind: slice
32 }
```

15.3 reflect.ValueOf()

`reflect.ValueOf()` 返回的是 `reflect.Value` 类型，其中包含了原始值的值信息。`reflect.Value` 与原始值之间可以互相转换。

```
1 package main
2
3 import (
4     "fmt"
```

```

5     "reflect"
6 )
7
8 func ReflectValue(x interface{}) {
9     v := reflect.ValueOf(x)
10    k := v.Kind()
11    switch k {
12    case reflect.Int:
13        fmt.Println(v.Int())
14    case reflect.Float32:
15        fmt.Println(v.Float())
16    case reflect.String:
17        fmt.Println(v.String())
18    case reflect.Bool:
19        fmt.Println(v.Bool())
20    default:
21        fmt.Println("unsupported type")
22    }
23 }
24
25 func main() {
26     var a float32 = 3.14
27     var b int = 100
28     ReflectValue(a) // 3.140000104904175
29     ReflectValue(b) // 100
30
31     var c string = "hello"
32     ReflectValue(c) // hello
33
34     // 将 int 类型的原始值转换为 reflect.Value 类型
35     var d = reflect.ValueOf(b)
36     fmt.Printf("Type d: %T\n", d) // Type d: reflect.Value
37 }

```

想要在函数中通过反射修改变量的值，需要注意函数参数传递的是值拷贝，必须传递变量地址才能修改变量值。而反射中使用专有的

Elem() 方法来获取指针对应的值。

```

1 package main
2
3 import (
4     "fmt"
5     "reflect"
6 )
7
8 func ReflectSetValue(x interface{}) {
9     v := reflect.ValueOf(x)
10    if v.Elem().Kind() == reflect.Int {
11        v.Elem().SetInt(123)
12    } else if v.Elem().Kind() == reflect.String {
13        v.Elem().SetString("hello")
14    } else {

```

```

15         fmt.Println("unsupported type")
16     }
17 }
18
19 func main() {
20     var a = 100
21     ReflectSetValue(&a)
22     fmt.Println(a) // 123
23
24     var b = "你好"
25     ReflectSetValue(&b)
26     fmt.Println(b) // hello
27 }

```

⚠ Caution

在使用 `Elem()` 之前，一定要判断 `Kind()`，否则非常容易触发 `Panic`。

15.4 结构体反射

15.4.1 与结构体相关的方法

任意值通过 `reflect.TypeOf()` 获得反射对象信息后，如果它的类型是结构体，可以通过反射值对象 `reflect.Type` 的 `NumField()` 和 `Field()` 方法获得结构体成员的详细信息。

- `Field(i int) StructField`：根据索引，返回索引对应的结构体字段的信息
- `NumField() int`：返回结构体成员字段数量。
- `FieldByName(name string) (StructField, bool)`：根据给定字符串返回字符串对应的结构体字段的信息。
- `FieldByIndex(index []int) StructField`：多层成员访问时，根据 `[]int` 提供的每个结构体的字段索引，返回字段的信息。
- `FieldByNameFunc(match func(string) bool) (StructField, bool)`：根据传入的匹配函数匹配需要的字段。
- `NumMethod() int`：返回该类型的方法集中方法的数量
- `Method(int) Method`：返回该类型方法集中的第 `i` 个方法
- `MethodByName(string) (Method, bool)`：根据方法名返回该类型方法集中的方法

15.4.2 StructField 类型

`StructField` 类型用来描述结构体中的一个字段的信息。`StructField` 的定义如下：

```

1  type StructField struct {
2      // 参见 http://golang.org/ref/spec#Uniqueness\_of\_identifiers
3      Name string // Name 是字段的名字
4      PkgPath string // PkgPath 是非导出字段的包路径，对导出字段该字段为""
5      Type Type // 字段的类型
6      Tag StructTag // 字段的标签
7      Offset uintptr // 字段在结构体中的字节偏移量
8      Index []int // 用于 Type.FieldByIndex 时的索引切片
9      Anonymous bool // 是否匿名字段
10 }

```

15.4.3 结构体反射实例

当我们使用反射得到一个结构体数据之后可以通过索引依次获取其字段信息，也可以通过字段名去获取指定的字段信息。

获取结构体属性：

```

1  package main
2
3  import (
4      "fmt"
5      "reflect"
6      "strconv"
7  )
8
9  type Student struct {
10     Name string `json:"name"`
11     Age  int    `json:"age"`
12     Score int   `json:"score"`
13 }
14
15 func (s *Student) GetInfo() string {
16     info := "Name:" + s.Name + " Age:" + strconv.Itoa(s.Age) + " Score:" +
17         strconv.Itoa(s.Score)
18     fmt.Println("GetInfo被调用:", info)
19     return info
20 }
21
22 func (s *Student) SetInfo(name string, age int, score int) {
23     s.Name = name
24     s.Age = age
25     s.Score = score
26     fmt.Println("SetInfo被调用")
27 }
28
29 func (s *Student) Print() {
30     fmt.Println("Print被调用: 打印。。。")
31 }
32
33 func PrintStructField(s interface{}) {
34     t := reflect.TypeOf(s)

```

```

34
35 // 处理指针情况：如果是指针，取其指向的类型
36 if t.Kind() == reflect.Ptr {
37     t = t.Elem()
38 }
39
40 if t.Kind() != reflect.Struct {
41     fmt.Println("It is not a struct")
42     return
43 }
44
45 // 1. 通过索引获取字段
46 field0 := t.Field(0)
47 fmt.Printf("字段0: 名称=%s, 类型=%s, JSON tag=%s\n", field0.Name, field0.Type,
48 field0.Tag.Get("json"))
49
50 // 2. 通过名称获取字段
51 if field1, ok := t.FieldByName("Name"); ok {
52     fmt.Printf("字段Name: 名称=%s, 类型=%s, JSON tag=%s\n", field1.Name,
53 field1.Type, field1.Tag.Get("json"))
54 }
55
56 // 3. 获取字段总数
57 fmt.Println("该结构体的字段数: ", t.NumField())
58 }
59
60 func PrintStructMethod(s interface{}) {
61     t := reflect.TypeOf(s)
62     v := reflect.ValueOf(s)
63
64     // 所以如果要调用指针接收者的方法，传入的必须是指针
65     if t.Kind() != reflect.Ptr || t.Elem().Kind() != reflect.Struct {
66         fmt.Println("Expected a pointer to a struct")
67         return
68     }
69
70     // 1. 通过类型变量里面的 Method 可以获取结构体的方法
71     // 1. 获取方法信息（按 ASCII 排序）
72     // 注意：确保至少有一个方法，否则 Method(0) 会 Panic
73     if t.NumMethod() > 0 {
74         var tMethod = t.Method(0)
75         fmt.Printf("第0个方法: 名称=%s, 类型=%s\n", tMethod.Name, tMethod.Type)
76     }
77
78     // 2. 获取方法总数
79     fmt.Println("该结构体的方法数: ", t.NumMethod())
80
81     fmt.Println(">>> 调用 Print 方法:")
82     mPrint := v.MethodByName("Print")
83     if mPrint.IsValid() {
84         mPrint.Call(nil)
85     } else {
86         fmt.Println("Print 方法不存在")
87     }
88 }

```

```

85     }
86
87     // 4. 执行方法传入参数
88     fmt.Println(">>> 调用 SetInfo 方法:")
89     mSetInfo := v.MethodByName("SetInfo")
90     if mSetInfo.IsValid() {
91         var params []reflect.Value
92         params = append(params, reflect.ValueOf("李四"))
93         params = append(params, reflect.ValueOf(22))
94         params = append(params, reflect.ValueOf(100))
95         mSetInfo.Call(params)
96     }
97
98     // 5. 执行方法获取返回值
99     fmt.Println(">>> 调用 GetInfo 方法:")
100    mGetInfo := v.MethodByName("GetInfo")
101    if mGetInfo.IsValid() {
102        results := mGetInfo.Call(nil)
103        // 打印返回值 (Results 是 []reflect.Value)
104        if len(results) > 0 {
105            fmt.Println("返回值:", results[0].String())
106        }
107    }
108 }
109
110 func main() {
111     stu1 := &Student{Name: "小王子", Age: 18, Score: 90}
112
113     // PrintStructField 可以传值也可以传指针（只要函数内处理好了）
114     // 这里演示传值
115     PrintStructField(*stu1)
116
117     // PrintStructMethod 必须传指针，因为 Student 的方法接收者是 *Student
118     PrintStructMethod(stu1)
119 }

```

修改结构体方法：

```

1  package main
2
3  import (
4      "fmt"
5      "reflect"
6  )
7
8  type Student struct {
9      Name  string `json:"name"`
10     Age   int    `json:"age"`
11     Score int    `json:"score"`
12 }
13
14 func (s Student) GetInfo() string {
15     return fmt.Sprintf("姓名：%v，年龄：%v，分数：%v", s.Name, s.Age, s.Score)

```

```

16 }
17
18 func reflectChangeStruct(s interface{}) {
19     t := reflect.TypeOf(s)
20     v := reflect.ValueOf(s)
21
22     if t.Elem().Kind() != reflect.Struct {
23         fmt.Println("It is not a struct")
24         return
25     }
26
27     name := v.Elem().FieldByName("Name")
28     name.SetString("李四")
29
30     age := v.Elem().FieldByName("Age")
31     age.SetInt(30)
32 }
33
34 func main() {
35     stu := &Student{
36         Name: "张三",
37         Age: 20,
38         Score: 90,
39     }
40
41     reflectChangeStruct(stu)
42     fmt.Println(stu.GetInfo()) // 姓名：李四，年龄：30，分数：90
43 }

```

Important

反射是一个强大并富有表现力的工具，能让我们写出更灵活的代码。但是反射不应该被滥用，原因有以下两个。

1. 基于反射的代码是极其脆弱的，反射中的类型错误会在真正运行的时候才会引发 `panic`，那很可能是在代码写完的很长时间之后。
2. 大量使用反射的代码通常难以理解。

16. IO 流

16.1 打开和关闭文件

`os.Open()` 函数能够打开一个文件，返回一个 `*File` 和一个 `err`。

Tip

操作完成文件对象以后一定要记得关闭文件


```

1 // 打开文件
2 file, err := os.Open("../.gitignore")
3 defer func(file *os.File) {
4     err := file.Close()
5     if err != nil {
6         return
7     }
8 }(file)
9 if err != nil {
10     return
11 }

```

16.2 读取文件

16.2.1 file.Read()

`file.Read()` 方法定义如下：

```

1 func (f *File) Read(b []byte) (n int, err error)

```

它接收一个字节切片，返回读取的字节数和可能的具体错误，读到文件末尾时会返回 0 和 `io.EOF`。

```

1 // 读取文件
2 var buf = make([]byte, 128) // 一次 128 字节
3 read, err := file.Read(buf)
4 if err == io.EOF {
5     fmt.Println("EOF")
6     return
7 }
8 if err != nil {
9     fmt.Println(err)
10    return
11 }
12 fmt.Printf("read: %d\n", read)
13 fmt.Println(string(buf[:read]))

```

16.2.2 循环读取

使用 `for` 循环读取文件中的所有数据

```

1 // 循环读取文件
2 var content []byte
3 var buffer = make([]byte, 128)
4
5 for {
6     read, err := file.Read(buffer)
7     if err == io.EOF {
8         fmt.Println("EOF")
9         break

```

```

10     }
11     if err != nil {
12         fmt.Println("Error:", err)
13         return
14     }
15     content = append(content, buffer[:read]...)
16 }
17 fmt.Println(string(content))

```

16.2.3 bufio.NewReader()

`bufio` 是在 `file` 的基础上封装了一层 API，支持更多的功能。

```

1  reader := bufio.NewReader(file)
2  for {
3      line, err := reader.ReadString('\n')
4      if err == io.EOF {
5          if len(line) != 0 {
6              fmt.Println(line)
7          }
8          fmt.Println("EOF")
9          break
10     }
11     if err != nil {
12         fmt.Println("Error:", err)
13         return
14     }
15     fmt.Println(line)
16 }

```

16.2.4 ioutil.ReadFile()

`io/ioutil` 包的 `ReadFile()` 方法能够读取完整的文件，只需要将文件名作为参数传入。

```

1  contents, err := ioutil.ReadFile("../.gitignore")
2  if err != nil {
3      fmt.Println("Error:", err)
4      return
5  }
6  fmt.Println(string(contents))

```

④ Note

`ioutil` 包已经在 Go 1.16 版本之后被废弃了

16.3 文件写入

`os.OpenFile()` 函数能够以指定模式打开文件， 从而实现文件写入相关功能。

```
1 func OpenFile(name string, flag int, perm FileMode) (*File, error) {
2     ...
3 }
```

其中：

- `name`： 要打开的文件名
- `flag`： 打开文件的模式。
- `perm`： 文件权限， 一个八进制数。 `r` (读) `04`, `w` (写) `02`, `x` (执行) `01`

16.3.1 file.Write()

```
1 // 创建一个文件
2 file, err := os.OpenFile(
3     "./demo34/test.txt", // 文件名
4     os.O_CREATE|os.O_RDWR, // 创建和读写模式
5     0666)
6
7 if err != nil {
8     return
9 }
10 defer func(file *os.File) {
11     err := file.Close()
12     if err != nil {
13         return
14     }
15 }(file)
16
17 contents := "你好 Golang\n"
18 _, err = file.Write([]byte(contents))
19 if err != nil {
20     return
21 } // 写入切片
22 _, err = file.WriteString("直接写入") // 写入字符串
23 if err != nil {
24     return
25 }
```

16.3.2 bufio.NewWriter()

```
1 writer := bufio.NewWriter(file)
2
3 _, err = writer.WriteString("\nHello World\n")
4 if err != nil {
5     return
6 }
7 err = writer.Flush()
8 if err != nil {
9     return
10 } // 将缓存中的内容写入文件
```

16.3.3 ioutil.WriteFile()

```
1 str := "hello world"
2 err = ioutil.WriteFile("./demo34/test.txt", []byte(str), 0666)
3 if err != nil {
4     return
5 }
```

17. 泛型

17.1 泛型的概念

Go 泛型的核心是类型参数 (Type Parameters) 和类型约束 (Type Constraints):

- **类型参数**: 用方括号 `[]` 声明在函数名或类型名之后, 代表一组可能的类型。
- **类型约束**: 限制类型参数的取值范围, 确保代码中对类型参数的操作是合法的

```
1 // 泛型函数
2 func 函数名[T 类型约束](参数 T) 返回值 T { ... }
3
4 // 泛型类型
5 type 类型名[T 类型约束] struct {
6     字段 T
7 }
```

Go 语言提供了两个内置的基础约束:

- **any**: 等价于 `interface{}`, 表示任意类型。
- **comparable**: 表示所有可比较的类型 (支持 `==` 和 `!=` 操作), 适用于需要比较操作的场景 (如 `map` 的 `key`)

```

1 package main
2
3 import "fmt"
4
5 // Swap 泛型函数：交换两个值
6 func Swap[T any](a, b *T) {
7     *a, *b = *b, *a
8 }
9
10 // 使用示例
11 func main() {
12     x, y := 10, 20
13     Swap(&x, &y) // 类型推断：无需显式指定T=int
14     fmt.Println(x, y) // 输出 20 10
15
16     s1, s2 := "hello", "world"
17     Swap(&s1, &s2) // 类型推断：T=string
18     fmt.Println(s1, s2) // 输出 world hello
19 }

```

17.2 类型约束

17.2.1 联合类型约束

使用 `|` 操作符指定多个允许的类型，适用于需要限制类型范围的场景：

```

1 // 支持int64和float64类型的求和函数
2 func SumNumbers[K comparable, V int64 | float64](m map[K]V) V {
3     var sum V
4     for _, v := range m {
5         sum += v
6     }
7     return sum
8 }

```

17.2.2 自定义接口约束

通过接口定义更复杂的类型约束，接口可以包含方法签名和类型集：

```

1 // 定义有序类型约束（支持 < 比较操作）
2 type Ordered interface {
3     int | int8 | int16 | int32 | int64 |
4     uint | uint8 | uint16 | uint32 | uint64 | uintptr |
5     float32 | float64 |
6     string
7 }
8
9 // 泛型函数：查找切片中的最大值
10 func Max[T Ordered](slice []T) T {
11     if len(slice) == 0 {

```

```

12     panic("slice is empty")
13 }
14 max := slice[0]
15 for _, v := range slice[1:] {
16     if v > max {
17         max = v
18     }
19 }
20 return max
21 }

```

17.2.3 近似类型约束

使用 `~` 操作符匹配底层类型为 `T` 的所有类型（包括自定义类型）：

```

1  type MyInt int
2
3  // 约束匹配所有底层类型为int的类型
4  type IntLike interface {
5      ~int
6  }
7
8  func Double[T IntLike](x T) T {
9      return x * 2
10 }
11
12 func main() {
13     var a int = 5
14     var b MyInt = 10
15     fmt.Println(Double(a)) // 输出 10
16     fmt.Println(Double(b)) // 输出 20 (MyInt底层是int)
17 }

```

17.3 类型推断

Go 语言泛型支持强大的**类型推断**机制，在大多数情况下可以省略类型参数，让代码更简洁：

17.3.1 函数参数类型推断

```

1  // 无需显式指定类型参数T
2  sum := SumNumbers(map[string]int64{"a": 1, "b": 2, "c": 3})
3  fmt.Println(sum) // 输出 6

```

17.3.2 赋值语句类型推断

```
1 // 从右侧推断Stack的类型参数为int
2 intStack := Stack[int]{}
3 // 等价于:
4 var intStack Stack[int]
```

17.3.3 方法调用类型推断

```
1 intStack.Push(1) // 从参数1推断T=int
2 element, ok := intStack.Pop() // 从返回值推断T=int
```

17.4 泛型接口

接口可以包含类型参数，定义更灵活的抽象：

```
1 type Parser[T any] interface {
2     Parse(data []byte) (T, error)
3     String() string
4 }
5
6 // 实现JSON解析器
7 type JSONParser[T any] struct{}
8
9 func (p JSONParser[T]) Parse(data []byte) (T, error) {
10     var result T
11     err := json.Unmarshal(data, &result)
12     return result, err
13 }
14
15 func (p JSONParser[T]) String() string {
16     return "JSON parser"
17 }
```